

Universidad de Valladolid

Práctica 2: Pruebas de carga, Locust y Prometheus

Grado en Ingeniería Informática
Evaluación de Sistemas Informáticos

Grupo 16

Índice

1. Introducción **3**
1.1. Descripción del sistema bajo prueba . . . 3

2. Estructura del proyecto **4**

3. Prometheus **5**
3.1. Instalación . . . 5

4. Métricas que vamos a emplear en la práctica **6**
4.1. Definición de las métricas que vamos a recoger . . . 6
4.2. Sintaxis de los contadores . . . 6
4.3. Decisiones importantes sobre *metrics.py* . . . 7

5. Modificaciones a *models.py* **8**
5.1. store_authorization_code . . . 8
5.2. consume_authorization_code . . . 8
5.3. issue_token_pair . . . 8
5.4. validate_refresh_token . . . 9
5.5. revoke_refresh_token . . . 9

6. Modificaciones a *app.py* **10**
6.1. Exposición del endpoint /metrics . . . 10
6.2. Registro automático de métricas HTTP . . . 10
6.3. Instrumentación de flujos OAuth2 — oauth2_routes.py . . . 11

7. Locust **13**
7.1. Instalación . . . 13
7.2. Diseño del fichero de pruebas de carga . . . 13
7.2.1. Endpoints objetivo del Locust . . . 13
7.2.2. Distribución del tráfico . . . 14
7.2.3. Tareas de tráfico legítimo . . . 14
7.2.4. Tareas de ataque . . . 15
7.2.5. Tareas de ataque EDR . . . 15
7.3. Decisiones de diseño relevantes . . . 15
7.4. Configuración de las pruebas . . . 16
7.5. Protección de la VM durante las pruebas . . . 16
7.6. Comportamientos del sistema frente a los dos escenarios de prueba . . . 17
7.6.1. Condiciones normales . . . 17
7.6.2. Condiciones de estrés . . . 18
7.6.3. Flujo de ejecución de las aplicaciones . . . 19

8. Verificación manual de métricas EDR **20**

9. Enfoque EDR del sistema de métricas **21**
9.1. Métricas adicionales implementadas . . . 21
9.1.1. Rendimiento del sistema . . . 22
9.1.2. Estado de tokens . . . 22
9.1.3. Detección de ataques . . . 23

10.Paneles de grafana **24**
10.1. Ataques de fuerza burta vs *credential stuffing* . . . 25
10.2. Ratio de Tokens . . . 25
10.3. Fallos del login de los usuarios . . . 26
10.4. Fallos del login de los usuarios . . . 26
10.5. Logins correctos en el sistema . . . 27
10.6. Intento de login por parte de usuarios desconocidos . . . 27
10.7. Peticiones a los diferentes endpoints de nuestro sistema . . . 28
10.8. Distribución de las peticiones en nuestro sistema . . . 28
10.9. Ratio peticiones correctas/fallidas por los usuarios . . . 29
10.10. Tiempo de respuesta a cada *endpoint* . . . 29
10.11Tipos de *tokens* en nuestro sistema. . . . 30
10.12Distribución de *tokens* en el sistema . . . 30
10.13Intentos de ataque de fuerza bruta . . . 31
10.14Flujos de autenticación en cada uno de los *endpoints* . . . 31

11.Distribución del trabajo **32**

Apéndices

A. Métricas del sistema en metrics.py **33**

B. app.py modificado **37**

C. Verificación manual con test.py **40**

D. Locust **52**

1 Introducción

En esta práctica estaremos empleando un servidor de autenticación *OAuth2*, el estándar por excelencia para hacer de mediación en la tarea de distribución de claves de acceso.
Sobre él, realizaremos pruebas de carga con Locust y recogeremos los resultados con Prometheus.
De tal manera que tenemos 3 objetivos:

1. Instrumentación del backend con métricas Prometheus (haremos hincapié en esta tecnología en el punto 3).
2. Despliegue y configuración de Prometheus y Grafana.
3. Pruebas de carga y estrés con Locust.

1.1 Descripción del sistema bajo prueba

El backend proporcionado por el profesor está desarrollado en Python con Flask y SQLite, implementando los tres flujos principales definidos en el RFC 6749:

- **Authorization Code**: flujo orientado a aplicaciones web donde un usuario humano se autentica mediante un formulario de login. El servidor emite un código temporal que posteriormente se canjea por un par de tokens.
- **Client Credentials**: flujo orientado a aplicaciones máquina a máquina. La aplicación se autentica directamente con sus credenciales y obtiene un *token* de acceso.
- **Refresh Token**: flujo que permite renovar un *token* de acceso caducado sin necesidad de repetir el proceso de autenticación completo.

La aplicación se gestiona con `uv`, el cual nosotros ya empleamos en nuestra primera práctica y como se explicó en su respectivo informe es un gestor de paquetes para Python (más moderno que el típico pip).
Aquí uv juega un papel fundamental, puesto que con el comando *uv add* instalará de manera automática las librerías y actualiza el .toml del proyecto para registra la dependencia en el *uv.lock*.

2 Estructura del proyecto

Para entender el proyecto vamos a ver qué hacen los ficheros que han sido proporcionados por el profesor y los que hemos tenido que crear para el mismo:

Fichero	Descripción
`app.py`	Punto de entrada de la aplicación Flask. Registra blueprints, configura la base de datos y expone el endpoint `/metrics`.
`oauth2_routes.py`	Implementación de los endpoints *OAuth2*: `/authorize` y `/token`.
`admin_routes.py`	Endpoints de administración bajo `/admin/clients` para gestionar clientes OAuth2.
`models.py`	Capa de acceso a datos: clientes, códigos de autorización y tokens.
`database.py`	Gestión de la conexión SQLite y definición del esquema.
`metrics.py`	Definición centralizada de todas las métricas Prometheus (fichero creado en esta práctica).
`seed.py`	Script de inicialización que crea dos clientes de prueba en la base de datos.

Cuadro 1: Estructura de ficheros modificados en el proyecto

De tal manera que el flujo de trabajo es el siguiente

`App OAuth2 (5001)` → `Prometheus (9090)` → `Grafana (3000)`

3 Prometheus

Prometheus es una base de datos de series temporales de tipo *pull*, es decir, es la propia base de datos quien periódicamente consulta las aplicaciones para conseguir las métricas.
Nuestra aplicación (*app.py*) toma un papel pasivo, donde se encarga de realizar sus tareas y de exponer un *endpoint* llamado */metrics* para que sea Prometheus el que coja los datos.

3.1 Instalación

Descargamos el binario de github y lo dejamos descomprimido en la VM

```
wget https://github.com/prometheus/prometheus/releases/download/\
v3.3.1/prometheus-3.3.1.linux-amd64.tar.gz
tar -xvf prometheus-3.3.1.linux-amd64.tar.gz
cd prometheus-3.3.1.linux-amd64
```

```
uv add prometheus-client
```

Para adaptarnos a las especificaciones de la práctica, hemos tenido que modificar el fichero *prometheus.yml* como mostramos a continuación.

```
global:
  #intervalo de busqueda de las metricas en el localhost 5001
  scrape_interval: 15s
  #intervalo de evaluacion de las reglas, para que nos llame grafana si se
     lia
  #no lo vamos a emplear en la practica
  evaluation_interval: 15s

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  #Esto es un nuevo trabajo que le dice al prometheus donde tiene que ir a
     buscar
  - job_name: "oauth2"
    static_configs:
      - targets: ["localhost:5001"]
```

Hemos definido el comportamiento del monitoreo, en el cual establecemos que se recojan métricas cada 15 segundos de la aplicación que está corriendo en el puerto 5001, en concreto de su endpoint */metrics*.
Ahora que lo tenemos todo configurado, simplemente ejecutamos el siguiente comando especificando su fichero de configuración.

```
./prometheus --config.file=prometheus.yml
```

4 Métricas que vamos a emplear en la práctica

Para esta sección, emplearemos 2 de los 4 tipos de métricas que nos proporciona la aplicación:

- ***Counter***: contador de incremento monotónico, esto quiere decir que sólo va a ir aumentando, lo usaremos para contar eventos como peticiones recibidas, tokens emitidos o errores producidos. Por convención, los counters tienen el sufijo `_total`.
- ***Gauge***: contador clásico, lo usaremos para valores que fluctúan como memoria usada o conexiones activas.

4.1 Definición de las métricas que vamos a recoger

Hemos creado un fichero *metrics.py*, detallado en el anexo A Métricas del sistema en metrics.py, en el que hemos añadido todos los contadores, con dos objetivos, por una parte centralizarlo todo en un fichero, facilitando el desarrollo y la depuración del código y por otra parte, evitar las importaciones circulares de módulos.
Este fenómeno ocurre cuando dos ficheros se importan entre ellos, es decir, si por ejemplo en *oauth2_ metrics.py* importamos cosas de *app.py* y en este último hacemos el *import* de funciones del primero, esto lo solucionamos gestionando todos los contadores en un mismo fichero.

4.2 Sintaxis de los contadores

Antes de mostrar todos nuestro trabajo en *metrics*, vamos a explicar un poco la sintaxis de este fichero, puesto que es una tecnología nueva, que al menos en la carrera no hemos tocado mucho:

- auth_flows_started = Counter(: nombre de variable la cual es un contador, la cual sólo puede incrementarse
- "oauth2_auth_flows_started_total": nombre de la métrica para su posterior integración con grafana
- "Flujos de autenticacion iniciados": descripción de la metrica
- ["*grant_type*"]: la etiqueta para luego poder filtrarlo en el grafana.)

Las **etiquetas** (*labels*) permiten que un mismo *counter* almacene valores independientes para cada combinación de etiquetas. Por ejemplo, `auth_flows_started` con la etiqueta `grant_type` almacena un contador separado para cada tipo de flujo:

```
oauth2_auth_flows_started_total{grant_type="client_credentials"}
   10.0
oauth2_auth_flows_started_total{grant_type="authorization_code"}
   3.0
oauth2_auth_flows_started_total{grant_type="refresh_token"} 5.0
```

Esta información nos fue útil en fases tempranas de la práctica ya que con un *curl* íbamos testeando las métricas para cerciorarnos de que realmente se estaban incrementando cuando deberían.

4.3 Decisiones importantes sobre *metrics.py*

Una decisión de diseño importante fue no añadir `client_id` en etiquetas como `tokens_issued`. Aunque inicialmente se contempló para tener información categorizadas por clientes, se descartó porque Prometheus crea una serie temporal por cada combinación única de etiquetas. Con una etiqueta de alta cardinalidad como `client_id` el número de series crecería proporcionalmente al número de clientes, lo que en una VM con 2GB de RAM podría causar problemas de memoria durante las pruebas de carga.

5 Modificaciones a *models.py*

models.py es la capa de acceso a datos. Se ha instrumentado con métricas en las siguientes funciones:

5.1 store_authorization_code

Se incrementa `pending_auth_codes` cada vez que se genera un código de autorización, reflejando que existe un código emitido pero no canjeado:

```
def store_authorization_code(...) -> str:
# ... codigo original ...
db.commit()
# A new code has been issued but not yet consumed
pending_auth_codes.inc()
return code
```

Listing 1: Instrumentación de store_authorization_code

5.2 consume_authorization_code

Se decrementa `pending_auth_codes` cuando el código se consume correctamente. Además detectamos dos casos de seguridad relevantes: el intento de reutilización de un código ya usado (`auth_code_reuse`) y el intento de uso de uno ya expirado:

```
def consume_authorization_code(code: str) -> Optional[dict]:
# ... codigo original ...
if data["used"]:
# Someone is trying to reuse an already consumed code
# This is a sign of an authorization code replay attack
auth_code_reuse.inc()
return None

if time.time() > data["expires_at"]:
return None

db.execute("UPDATE authorization_codes SET used = 1 ...")
db.commit()
# Code consumed correctly, no longer pending
pending_auth_codes.dec()
return data
```

Listing 2: Instrumentación de consume_authorization_code

5.3 issue_token_pair

Se incrementan los gauges de tokens activos cada vez que se emiten, distinguiendo entre *access* y *refresh tokens:*

```
def issue_token_pair(...) -> dict:
# ... codigo original ...
db.commit()
```

```
        # Track active tokens in the system
        active_tokens.labels(token_type="access").inc()
        if refresh_token:
        active_tokens.labels(token_type="refresh").inc()
        return response
```

Listing 3: Instrumentación de issue_token_pair

5.4 validate_refresh_token

Detectamos los dos casos de fallo relevantes para la seguridad: el intento de uso de un *token* revocado y el de uso de uno expirado.
Es importante que estos contadores se incrementen aquí en `models.py` y no en `oauth2_routes.py`, porque esta función tiene acceso directo al motivo exacto del fallo y nos evitamos meter ruido en las métricas:

```
        def validate_refresh_token(token: str) -> Optional[dict]:
        # ... codigo original ...
        if data["revoked"]:
        # Someone is using a token after it has been revoked
        # This is a sign of a token replay attack
        revoked_token_usage.inc()
        return None

        if time.time() > data["expires_at"]:
        # Someone is trying to use an expired token
        expired_token_usage.inc()
        return None

        return data
```

Listing 4: Instrumentación de validate_refresh_token

5.5 revoke_refresh_token

Incrementamos los *counters* de *tokens* revocados y se decrementan los *gauges* de *tokens* activos. La rotación de *tokens* del flujo `refresh_token` revoca tanto el *refresh token* antiguo como su *access token* asociado:

```
        def revoke_refresh_token(token: str) -> None:
        # ... codigo original ...
        db.commit()
        # Both the refresh token and its associated access token are revoked
        tokens_revoked.labels(token_type="refresh").inc()
        tokens_revoked.labels(token_type="access").inc()
        # Decrement active tokens accordingly
        active_tokens.labels(token_type="refresh").dec()
        active_tokens.labels(token_type="access").dec()
```

Listing 5: Instrumentación de revoke_refresh_token

6 Modificaciones a *app.py*

Modificamos `app.py` (código con las modificaciones detallado en el anexo B app.py modificado) para exponer el endpoint `/metrics` y registrar automáticamente todas las peticiones HTTP mediante decoradores Flask.

Los **decoradores** aplicados (`@before_request` y `@after_request`) son *hooks* de Flask que se ejecutan automáticamente antes y después de cada petición HTTP, obligando a cada uno de las peticiones a pasar por ellos.
Esto es lo que permite que nuestras métricas recojan los datos y todas las peticiones en un único lugar sin modificar cada ruta individualmente, siguiendo el principio DRY (*Don't Repeat Yourself*).
Además el `DispatcherMiddleware` combina la aplicación Flask con la mini aplicación de Prometheus que genera el endpoint `/metrics`, de modo que todo el tráfico a `/metrics` lo gestiona Prometheus y el resto lo gestiona Flask.

6.1 Exposición del endpoint /metrics

Se utiliza `DispatcherMiddleware` para combinar la aplicación Flask con Prometheus que genera el endpoint `/metrics`. Esto permite que pueda leer las métricas en `http://localhost:5001/metrics`:

```
from prometheus_client import make_wsgi_app
from werkzeug.middleware.dispatcher import DispatcherMiddleware
from werkzeug.serving import run_simple

if __name__ == "__main__":
flask_app = create_app()
# Combine Flask with the Prometheus /metrics endpoint
application = DispatcherMiddleware(flask_app, {
    "/metrics": make_wsgi_app()
})
run_simple("0.0.0.0", 5001, application,
use_reloader=True, use_debugger=True)
```

Listing 6: Exposición del endpoint /metrics

6.2 Registro automático de métricas HTTP

Hemos usado los decoradores `@before_request` y `@after_request` de Flask para registrar automáticamente todas las peticiones HTTP sin modificar cada ruta individualmente, para así seguir el patrón DRY (*Don't Repeat Yourself*):

```
@app.before_request
def before_request():
# Store arrival time and increment in-progress counter
g.start_time = time.time()
requests_in_progress.labels(endpoint=request.path).inc()

@app.after_request
def after_request(response):
endpoint = request.path

# General HTTP counter
```

```
            http_requests.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=str(response.status_code)
            ).inc()

            # Specific counters per endpoint
            if "/token" in endpoint:
            http_requests_token.labels(
            method=request.method,
            status_code=str(response.status_code)
            ).inc()
            elif "/authorize" in endpoint:
            http_requests_authorize.labels(
            method=request.method,
            status_code=str(response.status_code)
            ).inc()
            elif "/admin" in endpoint:
            http_requests_admin.labels(
            method=request.method,
            status_code=str(response.status_code)
            ).inc()

            # Decrement in-progress and record duration
            requests_in_progress.labels(endpoint=request.path).dec()
            duration = time.time() - g.start_time
            request_duration.labels(
            endpoint=request.path,
            method=request.method
            ).observe(duration)

            return response
```

Listing 7: Decoradores before y after request

6.3 Instrumentación de flujos OAuth2 — oauth2_routes.py

Hemos tenido que modificar este fichero puesto que, nuestras métricas afectan a los 3 flujos de nuestra aplicación.
De cara a esta modificación hemos seguido siempre la siguiente lógica: incrementamos `started` al entrar en la función, `failed` en cada punto de fallo y `success` junto con los contadores de tokens al completarse con éxito.
Esto se entiende mejor con un ejemplo:

```
            def _handle_client_credentials():
            auth_flows_started.labels(
            grant_type="client_credentials").inc()

            client_id, err = _authenticate_client()
            if err:
            auth_flows_failed.labels(
            grant_type="client_credentials").inc()
            return _error_response("invalid_client", err, 401)
```

```
10
11              client = get_client(client_id)
12              if client is None or \
13              "client_credentials" not in client["grant_types"]:
14              auth_flows_failed.labels(
15              grant_type="client_credentials").inc()
16              return _error_response("unauthorized_client",
17              "Client is not authorized for client_credentials")
18
19              scope = request.form.get("scope", "")
20              token_response = issue_token_pair(
21              client_id=client_id,
22              scope=scope,
23              user_id=None,
24              include_refresh=False,
25              )
26
27              auth_flows_success.labels(
28              grant_type="client_credentials").inc()
29              tokens_issued.labels(
30              token_type="access",
31              grant_type="client_credentials").inc()
32              access_tokens_issued.labels(
33              grant_type="client_credentials").inc()
34              # client_credentials no emite refresh token (RFC 6749)
35
36              return jsonify(token_response)
```

Listing 8: Fragmento de _handle_client_credentials()

Ahora que ya tenemos los dos primeros pasos configurados, vamos a preparar el grafana.
Lo primero que hacemos es añadir Prometheus como fuente de datos en *Connections* >*Data Sources* >*Add data source*, se configura la URL `http://localhost:9090` y se verifica la conexión con *Save & Test*, obteniendo el mensaje:

"Successfully queried the Prometheus API."

7 Locust

Llegados a este punto ya tenemos toda la parte de configurado de la aplicación hecho, pero claro no hemos mencionado nada de Locust, esto se debe a que Locust simplemente va a realizar las pruebas de carga que nosotros dictemos sobre nuestro sistema, por lo que necesitábamos todo el trabajo previo para que esta aplicación pudiese funcionar de manera correcta.

7.1 Instalación

Locust se instala como una dependencia del proyecto mediante `uv`:

```
cd ~/oauth2
uv add locust
```

Se arranca con el siguiente comando, que lanza la interfaz web en el puerto 8089:

```
uv run locust -f locustfile.py --host=http://localhost:5001
```

7.2 Diseño del fichero de pruebas de carga

En esta parte hemos dedicado mucho tiempo debido a que queríamos realizar una distribución realista del tráfico y no sabíamos muy bien como comprobar todas nuestras métricas desde un mismo punto. El fichero locust finalmente implementado se encuentra detallado en el anexo D Locust

7.2.1. Endpoints objetivo del Locust

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Avera size (byte
GET	/admin/clients	64	0	3600	4700	5000	3229.16	7	5026	290
POST	/authorize	477	0	3700	4800	5200	3552.18	23	5675	1225
GET	/authorize?client_id=web-app&redirect_uri=http%3A%2F%2Flocalhost%3A8080%2Fcallback&response_type=code&scope=read&state=locust-test	137	0	3600	4900	5000	3468.98	34	5414	3082
POST	/token	1140	0	3800	4900	5500	3610.83	7	5742	158.5
	Aggregated	1818	0	3700	4800	5300	3571.32	7	5742	663.3

Figura 1: Puntos objetivo de nuestras pruebas con el fichero de Locust

7.2.2. Distribución del tráfico

Tarea	Tipo	Peso	Probabilidad
`valid_client_credentials`	Legítimo M2M	6	27 %
`valid_authorization_code_flow`	Legítimo web	3	14 %
`valid_refresh_token_flow`	Legítimo renovación	2	9 %
`brute_force_api`	Ataque fuerza bruta	2	9 %
`brute_force_user_login`	Ataque fuerza bruta	2	9 %
`auth_code_replay_attack`	Ataque replay	1	5 %
`revoked_token_replay_attack`	Ataque replay	1	5 %
`invalid_refresh_token`	*Token* inválido	1	5 %
`client_enumeration_attack`	Ataque EDR	1	5 %
`credential_stuffing_attack`	Ataque EDR	1	5 %
`token_harvesting_attack`	Ataque EDR	1	5 %
`list_admin_clients`	Monitorización	1	5 %

Cuadro 2: Distribución de tareas del fichero Locust

Al querer hacer un diseño realista del trafico que soportaría un sistema como el nuestro hemos decidido emplear esta distribución teniendo en cuenta que cada usuario espera entre 1 y 3 segundos entre tareas mediante `wait_time = between(1, 3)`, para tener en cuenta ese "tiempo de pensar"que tienen los usuarios.
De tal manera que todo el tráfico de nuestra aplicación se divide en estas 3 categorías:

- **Tráfico legítimo**: 11/22 tareas = 50 %
- **Ataques**: 9/22 tareas = 41 %
- **Monitorización**: 1/22 tareas = 5 %

Siendo las tareas de administración las que menos peso tienen debido a que en un escenario real, un administrador de una web no se autentica varias veces ni hace un uso similar al de un usuario.

7.2.3. Tareas de tráfico legítimo

- `valid_client_credentials` **(peso 6)** es la tarea más frecuente porque el tráfico M2M es el mayoritario y el flujo más sencillo, puesto que el servidor solo se encarga de enviar las credenciales correctas al endpoint `/token`, verificando que la respuesta es un código 200.
- `valid_authorization_code_flow` **(peso 3)** implementa el flujo completo de autorización en dos pasos: primero realiza el login en `/authorize` obteniendo un código de autorización de la cabecera `Location` de la redirección 302, y después canjea ese código en `/token` por un par de tokens. Ojo esto es clave, ya que sin ella `pending_auth_codes` crecería indefinidamente al emitirse códigos que nunca se consumen.
- `valid_refresh_token_flow` **(peso 2)** obtiene primero un par de tokens mediante el flujo `authorization_code` y después los renueva usando el refresh *token*. Sin esta tarea las métricas `tokens_revoked` y `refresh_tokens_issued` siempre serían cero durante las pruebas de carga, ya que la rotación de tokens solo ocurre en este flujo.
- `list_admin_clients` **(peso 1)** consulta el endpoint de administración `/admin/clients` para generar tráfico en `http_requests_admin`, reflejando el bajo uso previsto del admin.

7.2.4. Tareas de ataque

- `brute_force_api` **(peso 2)** simula un ataque de fuerza bruta M2M enviando credenciales incorrectas contra el endpoint `/token`. Siempre usa el mismo `client_id` con *secrets* distintos.
- `brute_force_user_login` **(peso 2)** simula un ataque de fuerza bruta web enviando contraseñas incorrectas contra `/authorize`. Siempre usa el mismo username, lo que lo distingue del *credential stuffing* y dispara.
- `auth_code_replay_attack` **(peso 1)** canjea el mismo código de autorización dos veces. La primera petición tiene éxito (200) y la segunda falla(400) porque el código ya está marcado como usado en la base de datos, disparando la métrica `auth_code_reuse`.
- `revoked_token_replay_attack` **(peso 1)** simula un ataque de *token replay* en tres pasos: primero obtiene tokens válidos, los rota mediante el flujo `refresh_token` revocando el original, e intenta usar el *token* ya revocado. Dispara `revoked_token_usage`, señal de posible robo de tokens.
- `invalid_refresh_token` **(peso 1)** envía un *refresh token* completamente falso al servidor para ejercitar el camino de error de tokens inválidos y verificar que el servidor responde correctamente con un 400.

7.2.5. Tareas de ataque EDR

Estas tres tareas las hemos añadido específicamente para el enfoque EDR y detectar patrones de ataque:

- `client_enumeration_attack` **(peso 1)** prueba `client_id` comunes como `admin`, `root` o `test` seleccionados aleatoriamente de una lista de diez valores. Simula a un atacante escaneando qué clientes OAuth2 están registrados en el servidor, disparando la métrica `unknown_client_attempts`.
- `credential_stuffing_attack` **(peso 1)** selecciona aleatoriamente entre ocho pares de credenciales filtradas de otras plataformas y las prueba contra el endpoint `/authorize`. A diferencia de la fuerza bruta, cada intento usa un username distinto, lo que hace subir el *gauge* `unique_failed_usernames_active` y permite distinguir los dos tipos de ataque en el dashboard EDR.
- `token_harvesting_attack` **(peso 1)** solicita tokens válidos sin revocarlos ni renovarlos nunca. Hace que `active_tokens` suba sin que `tokens_revoked` lo acompañe proporcionalmente.

7.3 Decisiones de diseño relevantes

- `catch_response=True` se usa en todas las tareas de ataque para indicar a Locust que una respuesta 401 o 400 es el comportamiento esperado y no debe contabilizarse como fallo en las estadísticas de rendimiento. Sin esto Locust marcaría todos los ataques como errores y distorsionaría las métricas de la prueba.
- `allow_redirects=False` se usa en las peticiones a `/authorize` para capturar el código de autorización de la cabecera `Location` antes de que Locust siga la redirección automáticamente. Sin esto el código se perdería y el flujo `authorization_code` no podría completarse.
- **Método auxiliar** `_get_authorization_code()` encapsula el proceso de login y extracción del código de autorización. Es reutilizado por `valid_refresh_token_flow`, `auth_code_replay_attack`

y `revoked_token_replay_attack`, siguiendo el principio DRY y facilitando el mantenimiento del código.

- **Importaciones dentro de las tareas** (`import random`) se usan en los ataques de enumeración y credential *stuffing* para seleccionar aleatoriamente entre listas de valores, simulando comportamiento real de atacante y generando variedad en las etiquetas de las métricas de Prometheus.

7.4 Configuración de las pruebas

Desde la interfaz web de Locust vamos a configurar dos tipos de pruebas:

Prueba de carga — Evalúa el comportamiento del sistema en condiciones normales:

Parámetro	Valor
Number of users	50
Spawn rate	5 usuarios/segundo
Duración	5 minutos

Cuadro 3: Configuración de la prueba de carga normal dentro de un sistema de autenticación

Prueba de estrés — Evalúa el comportamiento del sistema en condiciones extremas para encontrar el límite real:

Parámetro	Valor
Number of users	200
Spawn rate	10 usuarios/segundo
Duración	3 minutos

Cuadro 4: Configuración de la prueba de estrés

7.5 Protección de la VM durante las pruebas

Debido a los recursos de nuestra VM, nos vemos con un problema similar al de la práctica 1, no podemos lanzar peticiones como locos sin preocuparnos de las consecuencias para nuestro sistema. Por ello hemos decidido tomar ciertas precauciones por lo que arrancamos prometheus con unos límites explícitos de retención de datos:

```
./prometheus --config.file=prometheus.yml \
--storage.tsdb.retention.time=2h \
--storage.tsdb.retention.size=500MB
```

Si el otro *dashboard* de grafana nos indica que estamos muy cerca de los límites, podemos llegar a lanzar estos comandos para tener un control a tiempo real sin necesidad de estar cambiando de panel constantemente:

```
watch -n 2 'echo "=== DISCO ===" && df -h / && \
echo "=== RAM ===" && free -h && \
echo "=== CPU ===" && top -bn1 | head -5'
```

7.6 Comportamientos del sistema frente a los dos escenarios de prueba

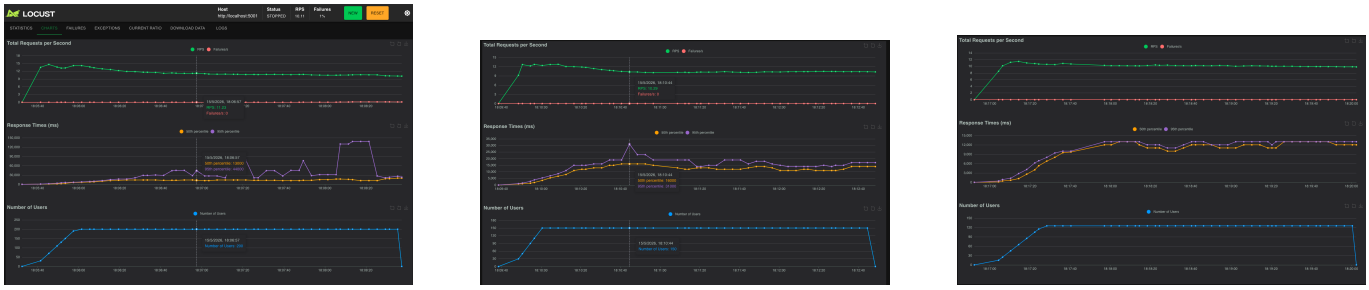
En este apartado mostraremos una captura general del locust en el que podremos comprobar cómo se ha comportado nuestro sistema ante las pruebas que hemos considerado adecuadas para sus dos escenarios.
Explicaremos en detalle cada uno de los paneles más adelante en este informe pero es una representación visual de cómo se está comportando el sistema.

7.6.1. Condiciones normales



Figura 2: Como podemos observar con los parámetros del locust del cuadro (3) obtenemos una resultados muy malos en el tiempo de respuesta, debido a las limitaciones de nuestra VM y diversos factores que explicaremos a continuación.

7.6.2. Condiciones de estrés



(a) 200 usuarios totales. (b) 150 usuarios totales. (c) 125 usuarios totales.

Figura 3: Pruebas de carga con diferentes usuarios totales partiendo del caso inical del cuadro 4

Como se puede observar en el panel de locust, ninguno de los intentos anteriores llegaba a tener un tiempo de respuesta razonable, con valores muy alejados de los 250ms que nos indicaban en el enunciado.
Con esto demostramos la existencia de un cuello de botella en la concurrencia de usuarios.

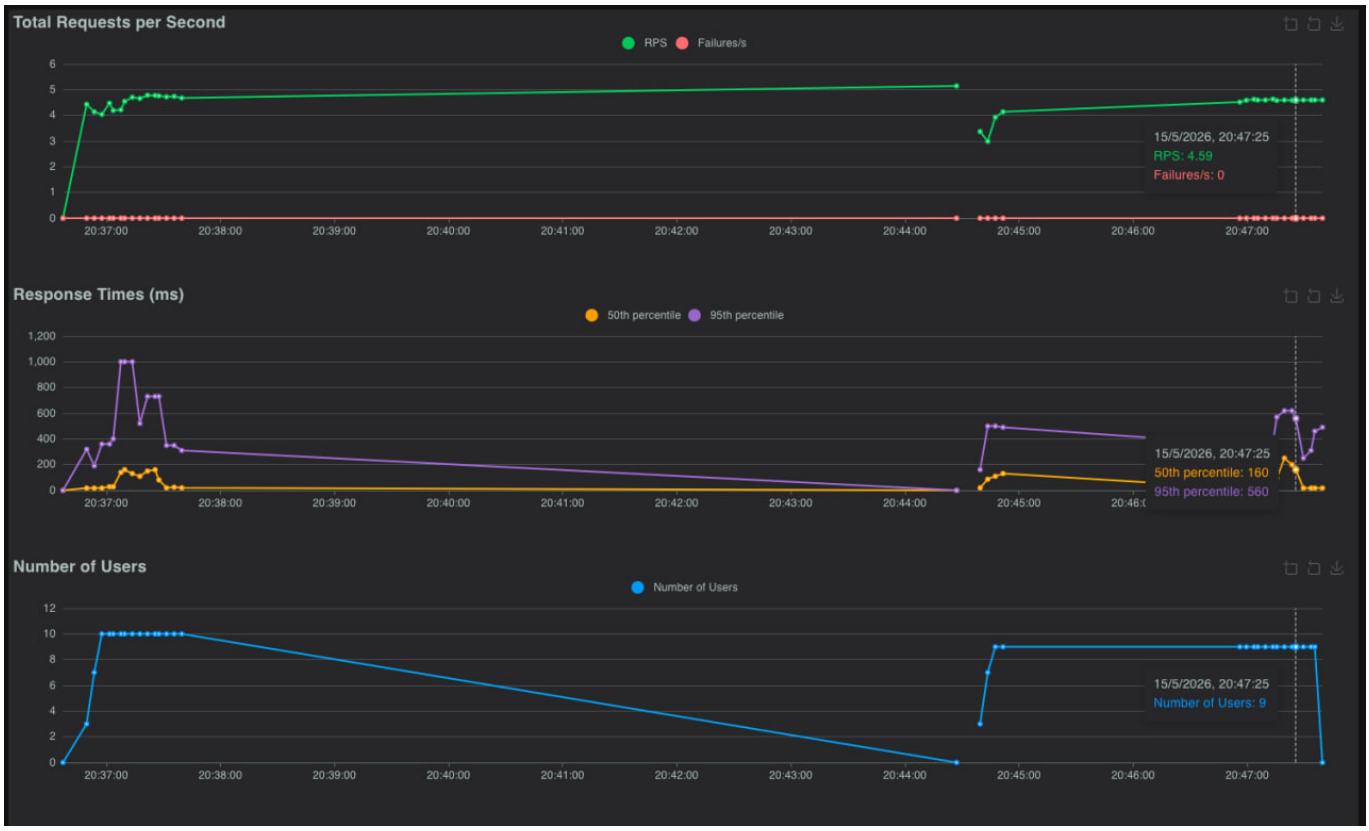


Figura 4: Pruebas con 10 y 9 usuarios,siendo el primer valor el límite de nuestro sistema sin que el tiempo de usuario tome valores inceptables. Estas métricas se han tomado el viernes, por lo que debido a la cantidad de personas conectadas a las VM pueden arrojar un rendimiento ligeramente peor del real.

En lo referente a la carga máxima del sistema, nos hemos percatado de dos problemas para llegar a ello, la primera de ellas es la nula paralelización de la máquina, contamos con dos cores, pero la aplicación no está preparada para emplearlos, de tal manera que toda la ejecución se realiza sobre un

único hilo y un core generando un cuello de botella importante para nuestro servidor y como atiende las peticiones, teniendo un límite de 12 rps lo que lastra el rendimiento de todo el sistema.
El segundo impedimento es nuestro *locustfile*, el cual está enfocado en peticiones muy pesadas debido a nuestro enfoque de EDR, por ello, es imposible llegar a las 250 ms de tiempo de respuesta que se nos marca como tiempo aceptable en el enunciado, debido a que ya con 2 usuarios, al tener que pasar por el proceso de cifrado para la obtención de los *tokens* el usuario que esté en ese proceso "secuestra"la CPU generando una cola de espera enorme.
En versiones anteriores de nuestro fichero de locust, versiones más ligeras, obteníamos mejores marcas de cargas de trabajo, pero esto se debía a que generaban un *fail fast* omitiendo algunas de las peticiones más pesadas y dando muestras "falsas" y ocultando el cuello de botella real de nuestro sistema, puesto que realmente estos ficheros de pruebas no estaban tan bien diseñados como el que finalmente hemos empleado en nuestra entrega.
Debido a esto, hemos decidido cambiar el tiempo entre peticiones para ver si eso ayudaba en algo, debido a que modificar la probabilidad de cada una de las tareas arruinaría por completo nuestra práctica y falsearía los resultados.
Por ello y tras realizar pruebas con 100, 25, 15, 10 usuarios, hemos llegado a la conclusión de que este último valor es el límite de nuestro sistema por lo puntos mencionados anteriormente.
El la captura del locust que adjuntamos, vemos la evolución del tiempo de respuesta con 10 y con 9 usuarios durante 3 minutos, tiempo más que suficiente para que ese número de usuarios se mantenga estable en el sistema y las métricas que nos arroja Locust sean realmente significativas.

7.6.3. Flujo de ejecución de las aplicaciones

1. Terminal 1:
   `uv run python app.py`
2. Terminal 2:
   `./prometheus -config.file=prometheus.yml`
3. Terminal 3:
   `uv run python test.py`
   `uv run locust -f locustfile.py -host=http://localhost:5001`
4. Terminal 4 (opcional, sólo si tenemos pocos recursos en la VM):
   `watch` para vigilancia de recursos

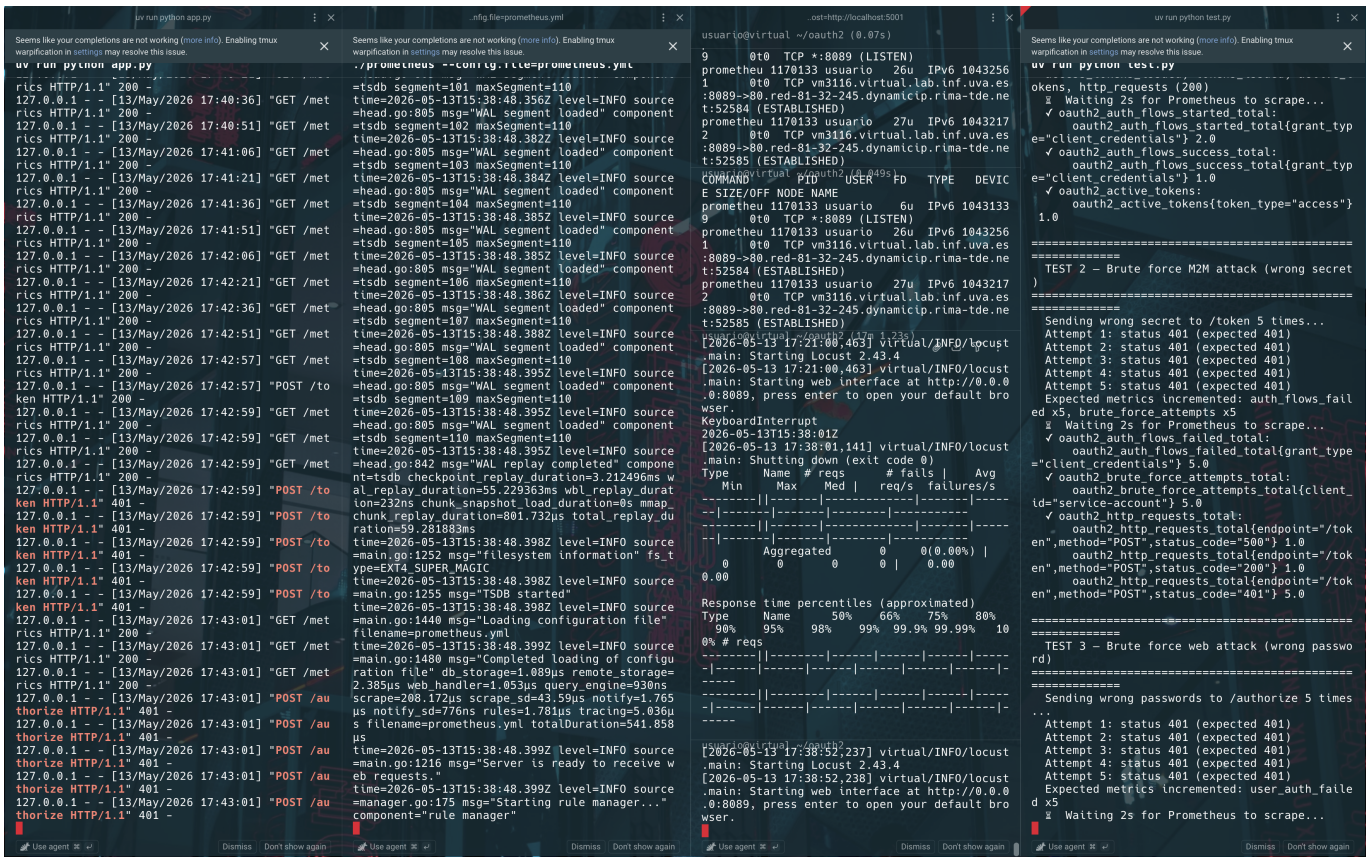


Figura 5: Ejecutamos la *app.py*, el locust y el prometheus, con el error de ejecución en los puertos erróneos

Y el orden de parada es el inverso, esperando 30 segundos entre Locust y Prometheus para que este último recoja los últimos datos antes de apagarse.

8 Verificación manual de métricas EDR

En el punto anterior hemos mencionado el `test.py`, el cual es un script que comprueba cada métrica de manera individual, en concreto comprobamos las siguientes:

Test	Nombre	Métricas verificadas
1	client_credentials válido	auth_flows_success, active_tokens
2	Fuerza bruta M2M	auth_flows_failed, brute_force_attempts
3	Fuerza bruta web	user_auth_failed
4	authorization_code completo	refresh_tokens_issued, pending_auth_codes
5	refresh_token válido	tokens_revoked, active_tokens
6	*Token* replay attack	revoked_token_usage
7	Auth code reuse	auth_code_reuse
8	Peticiones en paralelo	requests_in_progress, request_duration
9	*Token* falso/expirado	expired_token_usage

Cuadro 5: Tests del script de verificación manual

La comprobación se describe en detalle en el anexo, C Verificación manual con test.py

9 Enfoque EDR del sistema de métricas

El enunciado de la práctica señala explícitamente que uno de los objetivos de las pruebas de carga es identificar *"si surgen fallos de seguridad fruto del alto nivel de carga"* y que *"se podrán añadir métricas adicionales"*. Partiendo de este punto, hemos añadido un enfoque orientado a la ciberseguridad intentando emular un EDR (*Endpoint Detection and Response*), permitiendo detectar comportamientos anómalos en tiempo real durante las pruebas de carga.

Un sistema EDR monitoriza eventos de seguridad en tiempo real para detectar patrones de ataque como fuerza bruta, reutilización de tokens o intentos de acceso con credenciales comprometidas. Prometheus y Grafana permiten replicar este comportamiento de forma sencilla sobre el servidor OAuth2.

9.1 Métricas adicionales implementadas

Se han añadido tres categorías de métricas adicionales sobre las requeridas por el enunciado:

9.1.1. Rendimiento del sistema

Métrica	Descripción
`oauth2_request_duration_seconds`	*Histogram* del tiempo de respuesta por endpoint y método HTTP. Permite calcular el percentil 95 que el enunciado fija en 250ms como umbral de referencia.
`oauth2_requests_in_progress`	*Gauge* de peticiones HTTP simultáneas en curso en este momento. Útil para detectar ataques DoS bajo pruebas de estrés.

Cuadro 6: Métricas de rendimiento

9.1.2. Estado de tokens

Métrica	Descripción
oauth2_active_tokens	*Gauge* de tokens activos en el sistema por tipo. Sube al emitir tokens y baja al revocarlos.
oauth2_tokens_revoked_total	*Counter* de tokens revocados por tipo. Refleja la rotación de tokens en el flujo refresh_token.
oauth2_pending_auth_codes	*Gauge* de códigos de autorización emitidos pero no canjeados todavía. Un valor creciente indica clientes que piden códigos sin completar el flujo.
oauth2_revoked_token_usage_total	*Counter* de intentos de uso de tokens ya revocados. Señal de posible *token replay attack*.
oauth2_expired_token_usage_total	*Counter* de intentos de uso de tokens expirados.
oauth2_auth_code_reuse_total	*Counter* de intentos de reutilización de códigos de autorización ya consumidos. Señal de authorization code replay attack.

Cuadro 7: Métricas de estado de tokens

9.1.3. Detección de ataques

Métrica	Descripción
oauth2_brute_force_attempts_total	Counter de intentos fallidos de autenticación por client_id. Detecta ataques de fuerza bruta M2M.
oauth2_user_auth_failed_total	Counter de intentos fallidos de login por username y client_id. Detecta ataques de fuerza bruta sobre usuarios.

Cuadro 8: Métricas de detección de ataques

10 Paneles de grafana

Al igual que en la anterior entrega, hemos tenido que hacer un *dashboard* en grafana para tener una reperesentación gráfica de nuestro trabajo en esta práctica.



Figura 6: Aunque entraremos más en detalle, nuestro *dashboard* presenta dos tipos de metricas, las clásicas de pruebas de carga y las métricas específicas para nuestro enfoque EDR

Una vez que ya tenemos una idea de cómo hemos ditribuido el *dashboard* vamos a ir panel a panel explicando un poco la intención y alguna peculiaridad de la representación.
Las consultas con Prometheus son mucho más simples que las que nos tocó hacer con Postgres en la práctica anterior, pero al ser una herramienta nueva, nos ha llevado nuestro tiempo acostumbrarse a su sintaxis y sus operandos, teniendo que pedir consejo a Adrián sobre como escribir las peticiones para poder reflejar de una manera adecuada los datos.

10.1 Ataques de fuerza burta vs *credential stuffing*

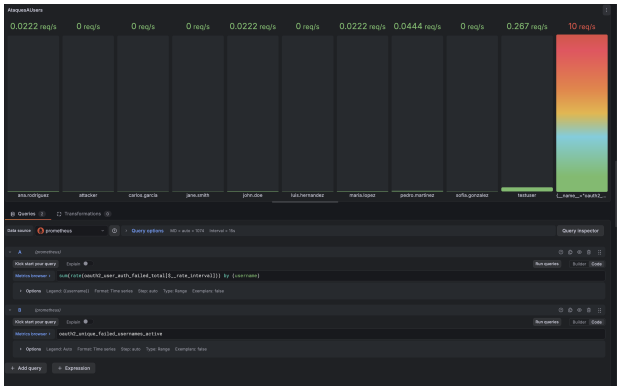


Figura 7: Con este panel estamos intentando diferenciar entre los errores de un usuario al escribir su contraseña con un posible ataque de fuerza bruta para entrar a la cuenta de un usuario.

Esta métrica pertenece a nuestro enfoque EDR, pero para entenderla bien, vamos a explicar cuáles son los dos tipos de ataques que estamos midiendo. Un *Brute Forze Attack* trata de una técnica de hacking que se basa en probar el acceso a la cuenta de un usuario empleando un diccionario de contraseñas, mientras que el *credential stuffing* se basa en intentar *loggearse* en un sistema empleando contraseñas de otros sitios de ese mismo usuario/mail.
Este panel muestra qué usuarios han sido atacados durante las pruebas, aquellos con más intentos fallidos son víctimas de fuerza bruta mientras que la presencia de muchos usuarios distintos con pocos intentos cada uno indica *credential stuffing*.

10.2 Ratio de Tokens

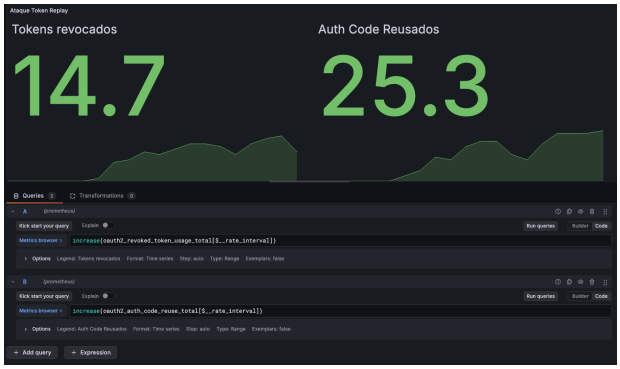


Figura 8: Tipos de tokens que maneja nuestro sistema

Con este panel lo que estamos intentando mirar es si estamos sufriendo ataques del tipo *Token Replay*, los cuales se basan en que un atacante ha conseguido un *token* válido y lo está empleando desde su cuenta para poder realizar ataques sobre nuestro sistema.
Para tener una visión limpia sobre este tipo de ataques estamos usando las fuunciones de incremento absoluto (*increase()*) en PromQL, las cuales revelan el número exacto de intentos de ataque neutralizados durante la ventana de evaluación. Además empleamos la métrica `oauth2_revoked_token_usage_total` para contabilizar las peticiones bloqueadas que intentan operar utilizando un *Refresh Token* que ya ha sido invalidado. ya que cualquier intento posterior de usarlas no solo es denegado, sino que queda registrado en este contador como una brecha de seguridad interceptada.

10.3 Fallos del login de los usuarios



Figura 9: Número de usuarios de nuestro sistema que han tenido fallos en su login

A simple vista, esta métrica parece un poco irrelevante ¿qué información útil aporta el número de usuarios que se han equivocado al introducir sus datos al hacer el *login*?, y la respuesta a esta pregunta es simple, un alto número de usuarios que fallan en su inicio de sesión es un indicativo de que algo anda mal, de tal manera que unido al panel anterior, nos da la información de que puede que uno de esos usuarios esté sufiendo un ataque y que por ende sea un vector de infección a nuestro sistema.

10.4 Fallos del login de los usuarios



Figura 10: Usuarios de nuestro sistema que han tenido fallos en su login

Métrica complementaria a la anterior, la cuál ya no sólo nos dice los intentos fallidos de *login en el sistema*, sino que tenemos una visión de qué usuarios están teniendo este comportamiento.
Decimos que es complementaria puesto que si el adminitrador del sistema detecta en el panel anterior un número grande de errores de login por parte de usuarios, consultado este panel puede ver qué usuario en concreto está teniendo este comportamiento, lo que permite contactar con él para que revise la actividad o en su defecto, cortarle el acceso al sistema por si te trata de un bot o un usuario que ha perdido el acceso a sus credenciales.
Así como dato extra la gráfica se construye mediante una consulta PromQL que calcula la tasa de fallos (*rate()*) y los agrupa por nombre de usuario (*sum(...) by (username)*), para poder realizar esta distinción.

10.5 Logins correctos en el sistema



Figura 11: Número de logins correctos en el sistema

Esta métrica es una clásica en lo que se refiere al monitoreo de un sistema y a las pruebas de carga. Con locust estamos testeando una carga de usuarios y peticiones simultáneas de tal manera que este panel nos sirve para ver cuántos usuarios están intentando acceder a nuestro sistema.

10.6 Intento de login por parte de usuarios desconocidos

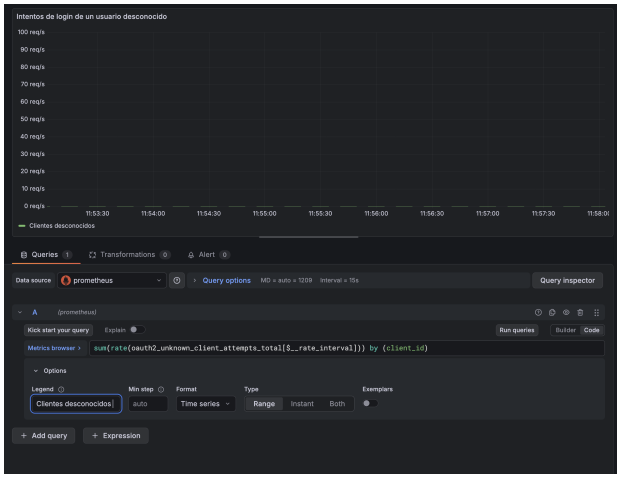


Figura 12: Peticiones de inicio de sesión de usuarios desconocidos para nuestro sistema.

Esta métrica es interesante, ya que estamos empleando un servidor de autenticación con una bbdd para registrar los usuarios con acceso a nuestros recursos y en caso de detectar algún incremento en esta métrica podríamos deducir que el sistema se ha visto comprometido o está siendo atacado desde un usuario del que no tenemos conocimiento. Adicionalmente casa muy bien con el enfoque que hemos decidido darle a la práctica ya que buscamos detectar posibles fallas cuando la carga de trabajo del sistema es muy grande.

10.7 Peticiones a los diferentes endpoints de nuestro sistema

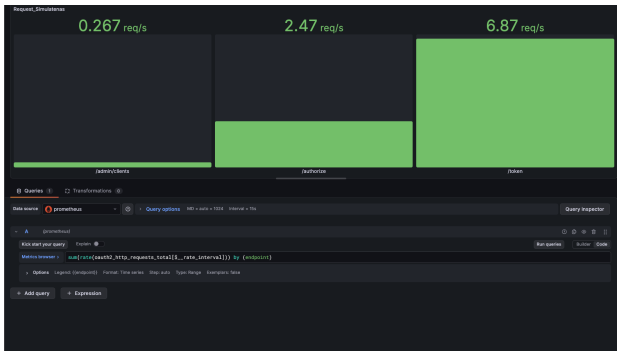


Figura 13: Peticiones que le llegan a los diferentes endpoints al sistema.

En este panel, nos encontramos con la distribución de peticiones a nuestro sistema.
Nos puede llamar la atención la distribución tan dispar de peticiones a cada uno de los *endpoints* pero hay que tener en cuenta que (`authorize`) es el endpoint que refleja una interacción humana, es decir, sería el formulario que seguiría un cliente al autenticarte en el sistema, por lo que, siguiendo el consejo de Adrián hemos generado una carga de trabajo lo más cercana a la realidad para ello.
Sin embargo el `/admin/clients` es la ruta de administración interna, la cual no debería estar expuesto a ataques del exterior, por lo que registra un tráfico residual y estable.
Hemos decidido implementar este panel para tener una información de la carga de trabajo en nuestro sistema, ya que podemos modificar los recursos destinados a cada uno o en caso de tener que tomar medidas saber exactamente la parte de nuestro sistema que está lastrando la *performance* del entorno.

10.8 Distribución de las peticiones en nuestro sistema

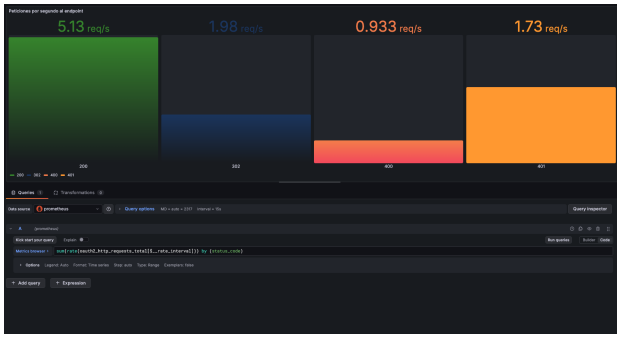


Figura 14: Distribución de las peticiones en nuestro sistema.

Panel que sirve para ver el estado de sistemas
El 200 es el código para las peticiones correctas, representado de color verde, mientras que las peticiones en el sector de color rojo agrupan todas las respuestas que no han resultado en un éxito, principalmente errores 400 de *Bad Request*, 401 de *Unauthorized* y posibles redirecciones 302, los cuales nos aportan una información muy relevante ya que los errores pueden implicar un mal funcionamiento del sistema.

10.9 Ratio peticiones correctas/fallidas por los usuarios



Figura 15: Tipos de tokens que maneja nuestro sistema

Nuestra idea con este panel es ver cómo se comportaba nuestro sistema ante los flujos de autenticación.
Como ya hemos comentado, nuestras pruebas no sólo buscan una prueba de carga sino comprobar la seguridad del sistema, por lo que este gráfico nos dice que el sistema es capaz de mantener un flujo constante de autorizaciones válidas pese a todo el ruido de peticiones maliciosas que le estamos haciendo, teniendo un comportamiento estable por lo que se puede ver en la linea de tendencias y que, además, no está dando falsos positivos. Es decir estamos ante un sistema robusto en lo referente a los *logeos*.

10.10 Tiempo de respuesta a cada *endpoint*

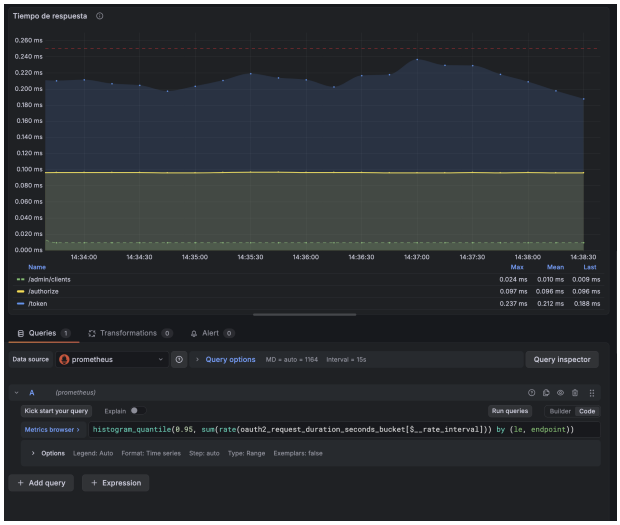


Figura 16: Gráfico que muestra el tiempo de respuesta a cada endpoint con una leyenda la cual nos da información exacta sobre los valores límite

Para entender la utilidad del panel tenemos que entender en la situación en la que estamos ejecutando este código.
Nuestro código se está ejecutando sobre el servidor de desarrollo integrado de Flask, y el entorno opera bajo un modelo de procesamiento síncrono. Cuando Locust inyecta un volumen de peticiones superior a la capacidad de despacho del servidor, el sistema operativo se ve obligado a encolar las peticiones entrantes en sus *buffers* de red (cola TCP), por este motivo, hay una discrepancia de tiempo entre lo que nos muestra el locust y lo mostrado en la gráfica, nosotros sólo estamos midiendo el tiempo de procesamiento de la petición mientras que locust nos da una visión más cercana al tiempo de usuario, desde que hace click y la web le devuelve el resultado de su petición.
La métrica `oauth2_request_duration_seconds` instrumentada en la aplicación Flask evalúa estrictamente el **tiempo de procesamiento interno**. Es decir, el sistema comienza a cronometrar en el milisegundo exacto en el que el servidor web acepta la conexión y entrega la petición al código Python, deteniéndose al emitir la respuesta. El código de la aplicación es inherentemente "ciego" al tiempo que la petición ha pasado esperando en etapas previas. De hecho para ser más precisos estamos empleando un percentil 95 para tener en cuenta el máximo rango posible de cara a la representación.

10.11 Tipos de *tokens* en nuestro sistema.



Figura 17: Información sobre tokens que hay actualmente en circulación en nuestro sistema.

Con este panel estamos registrando los tokens que se están emitiendo/empleando en nuestro sistema. Debemos recordar que estamos empleando un servidor oauth2 para la autenticación, por lo que esta estadística es necesaria para comprobar que el sistema funciona bien y que no estamos sufriendo ningún tipo de ataque.
Como tenemos 3 tipos vamos a explicar cada uno de ellos:

- **Query A (*Access Tokens*):** representa el número de sesiones activas que pueden realizar peticiones a recursos protegidos.
- **Query B (*Refresh Tokens*):** representa el número de sesiones de larga duración que permiten a los clientes obtener nuevas credenciales sin intervención del usuario.
- **Query C (*Total Tokens*):** como su nombre indica, la suma de las anteriores.

10.12 Distribución de *tokens* en el sistema

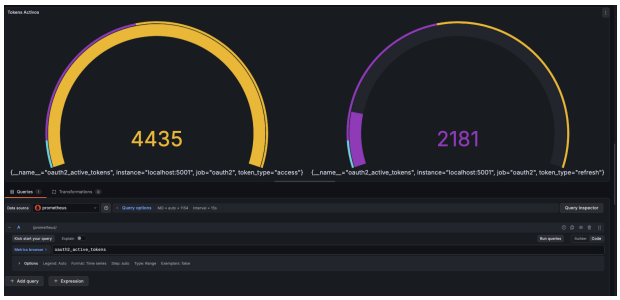


Figura 18: Información sobre todos los *tokens* emitidos en nuestro sistema.

Esta métrica es un histórico de todos los *tokens* emitidos en el sistema a nivel de acumulador
Si nos damos cuenta, esta métrica y la anterior tienen una relación, ya que una nos sirve de acumulador mientras que la anterior nos da una información de la producción de tokens y los tipos de cada uno.

10.13 Intentos de ataque de fuerza bruta

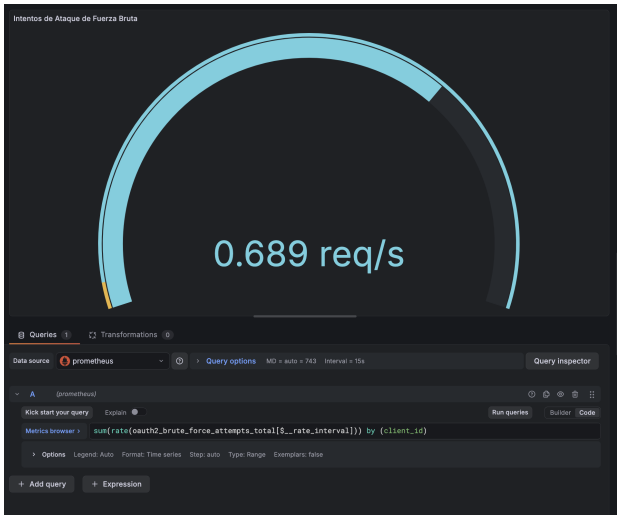


Figura 19: Peticiones consideradas de fuerza bruta que está recibiendo el sistema.

Esta métrica es vital para el enfoque EDR, puesto que nos da una información sobre un posible ataque al sistema.
Nuestro enfoque ha sido el siguiente, tenemos una métrica que se incrementa cuando un usuario falla el login, y debido a las condiciones del sistema no podemos tener un indicador más grande a puro código, pero si nos dirigimos al grafana, tenemos un panel para los intentos fallidos de login por usuario, lo que unido a un valor elevado de las *requests per second* en este panel nos permite aislar a aquellos usuarios que son problemáticos.

10.14 Flujos de autenticación en cada uno de los *endpoints*

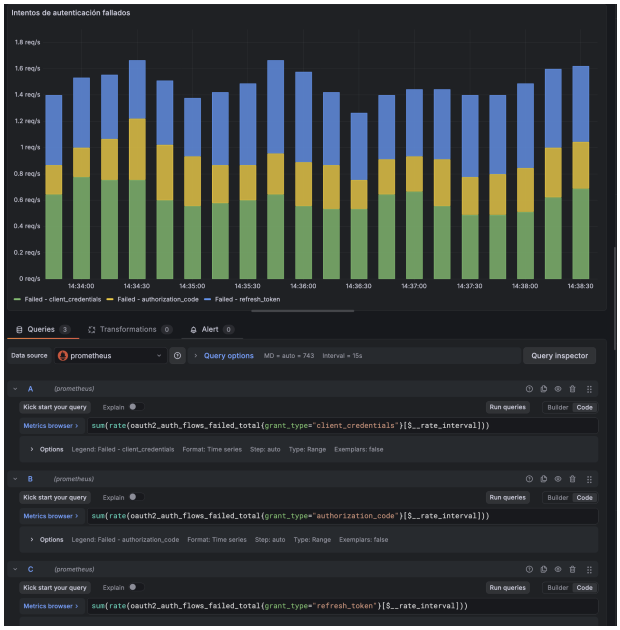


Figura 20: Tipos de tokens que maneja nuestro sistema

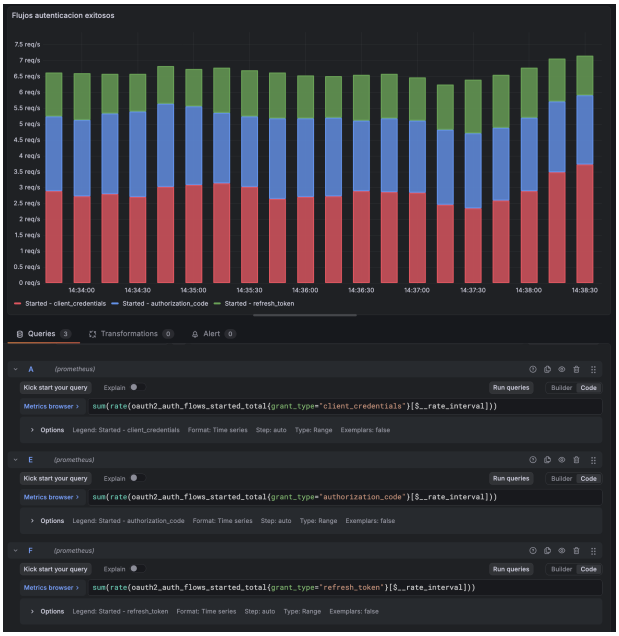


Figura 21: Tipos de tokens que maneja nuestro sistema

Para este panel lo que hemos buscado tener una representación visual de los intentos fallidos de inicio de sesión en cada uno de los *endpoints* de nuestro sistema.

- **Verde**: registra fallos en la autenticación máquina-a-máquina. Una tasa alta aquí indica intentos de fuerza bruta contra secretos de aplicaciones o cuentas de servicio.
- **Amarillo**: representa intentos fallidos de inicio de sesión en el formulario web, usualmente vinculados a ataques de diccionario o *credential stuffing*.

- **Azul:** contabiliza intentos de usar tokens de refresco inválidos, caducados o revocados. Es un indicador clave de ataques de *Replay* o de sesiones persistentes comprometidas.

11 Distribución del trabajo

Para la realización de esta práctica, nos hemos distribuido el trabajo encargándonos cada integrante de diferentes responsabilidades:

Nombre	**Porcentaje**	**Responsabilidad principal**
ÁLVAREZ FULCONIS, MARCOS P.	33 %	LocustFile, redacción de dicha parte en el documento y revisión de los esquemas de colores en el grafana
SANTIESTEBAN MENDO, EDGAR	33 %	Edicion del prometheus, de oauth_routes, models.py, redacción del documento
LOPEZ PRIMO, ÁLVARO	33 %	Creación de las métricas, modificación del app.py, enfoque EDR, dashboard grafana.

Cuadro 9: Distribución de Responsabilidades por Equipo y Miembro

A Métricas del sistema en metrics.py

```python
#Codigo para recopilar las metricas de Prometheus.

from prometheus_client import (
    Counter,
    Histogram,
    Gauge
)

#Flujos de autenticacion
# Contabiliza los intentos de llamada al endpoint de authorized o de token (exitosos o no)
auth_flows_started = Counter(
    "oauth2_auth_flows_started_total",
    "Flujos de autenticación iniciados",
    ["grant_type"] #permite almacenar el resultado de cada contador de manera
        independiente para poder tener una cifra de cada uno.
)

#Contabiliza los intentos exitosos de autenticacion en el sistema
auth_flows_success = Counter(
    "oauth2_auth_flows_success_total",
    "Flujos de autenticación completados con éxito",
    ["grant_type"]
)

#Contabiliza los intentos fallidos de autenticacion en el sistema
auth_flows_failed = Counter(
    "oauth2_auth_flows_failed_total",
    "Flujos de autenticación fallidos",
    ["grant_type"]
)

#Tokens emitidos por la aplicacion a nivel general
tokens_issued = Counter(
    "oauth2_tokens_issued_total",
    "Tokens emitidos",
    ["token_type", "grant_type"]
)

#Tokens emitidos (separados por tipo)
access_tokens_issued = Counter(
    "oauth2_access_tokens_issued_total",
    "Access tokens emitidos",
    ["grant_type"]
)
refresh_tokens_issued = Counter(
    "oauth2_refresh_tokens_issued_total",
    "Refresh tokens emitidos",
    ["grant_type"]
)
```

```
49
50 #Llamadas HTTP para medir el numero de peticiones a nivel general
51 http_requests = Counter(
52     "oauth2_http_requests_total",
53     "Número de llamadas HTTP",
54     ["method", "endpoint", "status_code"]
55 )
56
57 #Llamadas HTTP (separadas por endpoint)
58 http_requests_token = Counter(
59     "oauth2_http_requests_token_total",
60     "Llamadas al endpoint /token",
61     ["method", "status_code"]
62 )
63
64 http_requests_authorize = Counter(
65     "oauth2_http_requests_authorize_total",
66     "Llamadas al endpoint /authorize",
67     ["method", "status_code"]
68 )
69
70 http_requests_admin = Counter(
71     "oauth2_http_requests_admin_total",
72     "Llamadas a los endpoints /admin",
73     ["method", "status_code"]
74 )
75
76 #Permite la medicion del numero de request que estan en proceso en este momento
77 requests_in_progress = Gauge(
78     "oauth2_requests_in_progress",
79     "Peticiones HTTP en curso en este momento",
80     ["endpoint"]
81 )
82
83 #Metricas extra para el planteamiento de ser un EDR
84 #Con esto podemos medir el tiempo medio de duracion de un request
85 request_duration = Histogram(
86     "oauth2_request_duration_seconds",
87     "Tiempo de respuesta de cada peticion",
88     ["endpoint", "method"],
89     # 0.25 es el umbral de 250ms que marca el enunciado como referencia
90     buckets=[ 0.001,
91         0.005,
92         0.01,
93         0.025,
94         0.05,
95         0.1,
96         0.25,
97         0.5,
98         1.0,
99         2.5,
100        5.0,
101        10.0]
```

```
)

#Con esto intentamos ver si esta pasando algo raro con los tokens
#Info sobre los tokens activos en el sistema
active_tokens = Gauge(
    "oauth2_active_tokens",
    "Tokens de acceso activos en el sistema",
    ["token_type"]
)

#Info sobre los tokens caducados
tokens_revoked = Counter(
    "oauth2_tokens_revoked_total",
    "Tokens revocados",
    ["token_type"]
)

#Nos da informacion sobre los codigos emitidos pero no empleados
pending_auth_codes = Gauge(
    "oauth2_pending_auth_codes",
    "Codigos de autorizacion emitidos pero no canjeados"
)

# Intentos de uso de tokens ya revocados
# Señal de posible token replay attack
revoked_token_usage = Counter(
    "oauth2_revoked_token_usage_total",
    "Intentos de uso de tokens ya revocados"
)

# Intentos de uso de tokens expirados
expired_token_usage = Counter(
    "oauth2_expired_token_usage_total",
    "Intentos de uso de tokens expirados"
)

# Intentos de reutilizacion de codigos de autorizacion ya usados
# Señal de posible authorization code replay attack
auth_code_reuse = Counter(
    "oauth2_auth_code_reuse_total",
    "Intentos de reutilizacion de codigos ya usados"
)

# Intentos fallidos de autenticacion por client_id
# Util para detectar ataques de fuerza bruta contra un cliente concreto
brute_force_attempts = Counter(
    "oauth2_brute_force_attempts_total",
    "Intentos fallidos de autenticacion por cliente",
    ["client_id"]
)

# Intentos fallidos de login de usuario
# Util para detectar ataques de fuerza bruta contra usuarios concretos
```

```
155 user_auth_failed = Counter(
156     "oauth2_user_auth_failed_total",
157     "Intentos fallidos de autenticacion de usuario",
158     ["username", "client_id"]
159 )
160
161 # Operaciones de gestion de clientes OAuth2
162 clients_created = Counter(
163     "oauth2_admin_clients_created_total",
164     "Clientes OAuth2 creados via API admin"
165 )
166
167 clients_deleted = Counter(
168     "oauth2_admin_clients_deleted_total",
169     "Clientes OAuth2 eliminados via API admin"
170 )
171
172 clients_listed = Counter(
173     "oauth2_admin_clients_listed_total",
174     "Consultas al listado de clientes"
175 )
176
177 user_auth_success = Counter(
178     "oauth2_user_auth_success_total",
179     "Inicios de sesion exitosos por usuario",
180     ["username"]
181 )
182
183 #Vamos a simular el comportamiento de un usuario
184 #metrica para los clicks del formulario.
185 login_page_visits = Counter(
186     "oauth2_login_page_visits_total",
187     "Visitas a la pagina de login por client_id",
188     ["client_id"]
189 )
190 #Intentos de login con un usario desconocido
191 unknown_client_attempts = Counter(
192     "oauth2_unknown_client_attempts_total",
193     "Intentos de acceso con client_id desconocido",
194     ["client_id"]
195 )
196
197 # Un valor alto indica credential stuffing
198 # Un valor bajo con muchos fallos indica fuerza bruta clasica
199 unique_failed_usernames = Gauge(
200     "oauth2_unique_failed_usernames_active",
201     "Numero de usernames distintos con fallos de login activos",
202 )
```

B app.py modificado

```python
"""
app.py --Flask application entry point.

Wires together the database, blueprints, and configuration.
Run with: uv run python app.py
"""
#DISCLAIMER we've commented in english to mantain the consistence of the code

#request had the request info
#g will be use to store the request live time
from flask import Flask, request, g
#wsi will be the way prometheus gets their data
from prometheus_client import make_wsgi_app
#use for the comunication btw the wsgi and metrics.py
from werkzeug.middleware.dispatcher import DispatcherMiddleware
import time

from database import close_db, init_db
from oauth2_routes import oauth2
from admin_routes import admin
#we need this lib for the metrics that we've already defined on metrics.py
from metrics import (
       http_requests,
       http_requests_token,
       http_requests_admin,
       http_requests_authorize,
       request_duration,
       requests_in_progress
)

from werkzeug.serving import run_simple
# ---------------------------------------------------------------------------
# Hardcoded test users (resource owners for the authorization code flow).
# In production these would live in a proper user store.
# ---------------------------------------------------------------------------
USERS = {
    "testuser": "testpass",
    "admin": "adminpass",
}


def create_app() -> Flask:
    app = Flask(__name__)
    app.secret_key = "dev-secret-key-change-in-production"

    # Store users in app config so routes can access them
    app.config["USERS"] = USERS

    # Register blueprints
```

```
    app.register_blueprint(oauth2)
    app.register_blueprint(admin)

    # Database lifecycle
    app.teardown_appcontext(close_db)

    # Ensure tables exist on first launch
    with app.app_context():
        init_db()

    #Automatic HTTP metrics log
    # the Flask decorator
    @app.before_request
    def before_request():
        # Store the moment of the request arrives
        g.start_time = time.time()
        requests_in_progress.labels(endpoint=request.path).inc()
    #This force that every request must execute this methods
    @app.after_request
    def after_request(response):
        # Get the request endpoint
        endpoint = request.path

        # We increment the general counter
        http_requests.labels(
            method=request.method,
            endpoint=endpoint,
            status_code=str(response.status_code)
        ).inc()

        # We increment the specific counter for each endpoint
        if "/token" in endpoint:
            http_requests_token.labels(
                method=request.method,
                status_code=str(response.status_code)
            ).inc()
        elif "/authorize" in endpoint:
            http_requests_authorize.labels(
                method=request.method,
                status_code=str(response.status_code)
            ).inc()
        elif "/admin" in endpoint:
            http_requests_admin.labels(
                method=request.method,
                status_code=str(response.status_code)
            ).inc()
        requests_in_progress.labels(endpoint=request.path).dec()
        # Calc and store the request duration
        duration = time.time() - g.start_time
        request_duration.labels(
            endpoint=request.path,
            method=request.method
        ).observe(duration)
```

```
        return response

    return app


if __name__ == "__main__":
    # Cretaion of the Flask app
    flask_app = create_app()

    # Merge Flask with the /metrics (Prometheus) endpoint
    application = DispatcherMiddleware(flask_app, {
        "/metrics": make_wsgi_app()
    })

    print("OAuth2 server running on http://localhost:5001")
    print("Metrics available on http://localhost:5001/metrics")
    print("Endpoints:")
    print(" GET /authorize --Authorization Code flow (browser)")
    print(" POST /token --Token endpoint")
    print(" GET /admin/clients --List all clients")
    print(" POST /admin/clients --Register a new client")
    print(" DELETE /admin/clients/<id> --Delete a client")
    print()

    run_simple("0.0.0.0", 5001, application, use_reloader=True, use_debugger=True)
```

C Verificación manual con test.py

```python
"""
test_metrics.py --Manual verification script for all Prometheus metrics.

This script tests every metric defined in metrics.py by simulating:
    - Legitimate traffic (client_credentials, authorization_code, refresh_token)
    - Brute force attacks (wrong credentials, wrong passwords)
    - Security attacks (token replay, auth code reuse)
    - Parallel requests (to trigger requests_in_progress)

Usage:
    uv run python test_metrics.py

Requirements:
    - app.py must be running on localhost:5001
    - seed.py must have been executed previously

Author: Grupo 16
"""

import requests
import threading
import time

# --Configuration --------------------------------------------------------------
BASE_URL = "http://localhost:5001"
METRICS_URL = f"{BASE_URL}/metrics"

# Credentials from seed.py
SERVICE_ACCOUNT_ID = "service-account"
SERVICE_ACCOUNT_SECRET = "6Q6TZj_hKy4xBnZhgBuIwdJ6lSaQbF-9XWR9S_BHthk"
WEB_APP_ID = "web-app"
WEB_APP_SECRET = "Vh_lxCQeixB4jAraZRxcd0kmP43YOeyXAn7IVomdMmc"

# Valid test users (defined in app.py)
VALID_USERNAME = "testuser"
VALID_PASSWORD = "testpass"


# --Helpers --------------------------------------------------------------------

def separator(title: str) -> None:
    """Print a section separator."""
    print(f"\n{'='*60}")
    print(f" {title}")
    print(f"{'='*60}")


def check_metric(metric_name: str) -> None:
    """Query /metrics and print the value of a specific metric."""
```

```
    response = requests.get(METRICS_URL)
    lines = [
        line for line in response.text.splitlines()
        if metric_name in line and not line.startswith("#")
    ]
    if lines:
        print(f" todo correcto con {metric_name}:")
        for line in lines:
            print(f" {line}")
    else:
        print(f" fallo en {metric_name}: no value yet (metric not incremented)")


def wait_for_prometheus(seconds: int = 2) -> None:
    """Wait for Prometheus to scrape the latest metrics."""
    print(f" Waiting {seconds}s for Prometheus to scrape...")
    time.sleep(seconds)


# --Test blocks -----------------------------------------------------------

def test_client_credentials_valid() -> str | None:
    """
    Tests a valid client_credentials request.
    Expected metrics incremented:
        - auth_flows_started{grant_type="client_credentials"}
        - auth_flows_success{grant_type="client_credentials"}
        - access_tokens_issued{grant_type="client_credentials"}
        - tokens_issued{token_type="access", grant_type="client_credentials"}
        - active_tokens{token_type="access"}
        - http_requests{method="POST", endpoint="/token", status_code="200"}
    Returns the access_token for later use.
    """
    separator("TEST 1 --Valid client_credentials request")
    print(" Sending correct credentials to /token...")

    response = requests.post(f"{BASE_URL}/token", data={
        "grant_type": "client_credentials",
        "client_id": SERVICE_ACCOUNT_ID,
        "client_secret": SERVICE_ACCOUNT_SECRET,
        "scope": "read"
    })

    print(f" Response status: {response.status_code}")
    if response.status_code == 200:
        token = response.json().get("access_token")
        print(f" Access token received: {token[:20]}...")
        print(" Expected metrics incremented: auth_flows_started, auth_flows_success,")
        print(" access_tokens_issued, tokens_issued, active_tokens, http_requests (200)")
        wait_for_prometheus()
        check_metric("oauth2_auth_flows_started_total")
        check_metric("oauth2_auth_flows_success_total")
        check_metric("oauth2_active_tokens")
```

```
        return token
    else:
        print(f" ERROR: expected 200 but got {response.status_code}")
        return None


def test_client_credentials_invalid() -> None:
    """
    Tests a client_credentials request with wrong secret.
    Expected metrics incremented:
        - auth_flows_started{grant_type="client_credentials"}
        - auth_flows_failed{grant_type="client_credentials"}
        - brute_force_attempts{client_id="service-account"}
        - http_requests{method="POST", endpoint="/token", status_code="401"}
    """
    separator("TEST 2 --Brute force M2M attack (wrong secret)")
    print(" Sending wrong secret to /token 5 times...")

    for i in range(5):
        response = requests.post(f"{BASE_URL}/token", data={
            "grant_type": "client_credentials",
            "client_id": SERVICE_ACCOUNT_ID,
            "client_secret": f"wrong_secret_{i}"
        })
        print(f" Attempt {i+1}: status {response.status_code} (expected 401)")

    print(" Expected metrics incremented: auth_flows_failed x5, brute_force_attempts x5")
    wait_for_prometheus()
    check_metric("oauth2_auth_flows_failed_total")
    check_metric("oauth2_brute_force_attempts_total")
    check_metric("oauth2_http_requests_total")


def test_user_login_invalid() -> None:
    """
    Tests failed user login attempts via /authorize.
    Expected metrics incremented:
        - user_auth_failed{username="attacker", client_id="web-app"}
        - http_requests{method="POST", endpoint="/authorize", status_code="401"}
    """
    separator("TEST 3 --Brute force web attack (wrong password)")
    print(" Sending wrong passwords to /authorize 5 times...")

    for i in range(5):
        response = requests.post(f"{BASE_URL}/authorize", data={
            "client_id": WEB_APP_ID,
            "redirect_uri": "http://localhost:8080/callback",
            "response_type": "code",
            "scope": "read",
            "state": "test123",
            "action": "approve",
            "username": "attacker",
            "password": f"wrong_password_{i}"
```

```
        })
        print(f" Attempt {i+1}: status {response.status_code} (expected 401)")

    print(" Expected metrics incremented: user_auth_failed x5")
    wait_for_prometheus()
    check_metric("oauth2_user_auth_failed_total")


def test_authorization_code_flow() -> tuple[str | None, str | None]:
    """
    Tests the complete authorization_code flow.
    Step 1: Login with correct credentials to get an auth code.
    Step 2: Exchange the auth code for tokens.
    Expected metrics incremented:
        - auth_flows_success{grant_type="authorization_code"}
        - access_tokens_issued{grant_type="authorization_code"}
        - refresh_tokens_issued{grant_type="authorization_code"}
        - pending_auth_codes (inc then dec)
        - active_tokens{token_type="access"}
        - active_tokens{token_type="refresh"}
    Returns (access_token, refresh_token) for later use.
    """
    separator("TEST 4 --Valid authorization_code flow")

    # Step 1: Login to get the auth code
    print(" Step 1: Logging in to get authorization code...")
    response = requests.post(f"{BASE_URL}/authorize",
        data={
            "client_id": WEB_APP_ID,
            "redirect_uri": "http://localhost:8080/callback",
            "response_type": "code",
            "scope": "read",
            "state": "test123",
            "action": "approve",
            "username": VALID_USERNAME,
            "password": VALID_PASSWORD
        },
        allow_redirects=False # we need to capture the redirect to extract the code
    )

    print(f" Step 1 status: {response.status_code} (expected 302)")

    if response.status_code != 302:
        print(f" ERROR: expected 302 but got {response.status_code}")
        return None, None

    # Extract the auth code from the redirect URL
    location = response.headers.get("Location", "")
    print(f" Redirect URL: {location}")

    if "code=" not in location:
        print(" ERROR: no code found in redirect URL")
        return None, None
```

```
209
210     auth_code = location.split("code=")[1].split("&")[0]
211     print(f" Authorization code obtained: {auth_code[:20]}...")
212
213     # Step 2: Exchange the auth code for tokens
214     print(" Step 2: Exchanging auth code for tokens...")
215     response = requests.post(f"{BASE_URL}/token", data={
216         "grant_type": "authorization_code",
217         "code": auth_code,
218         "redirect_uri": "http://localhost:8080/callback",
219         "client_id": WEB_APP_ID,
220         "client_secret": WEB_APP_SECRET
221     })
222
223     print(f" Step 2 status: {response.status_code} (expected 200)")
224
225     if response.status_code == 200:
226         data = response.json()
227         access_token = data.get("access_token")
228         refresh_token = data.get("refresh_token")
229         print(f" Access token: {access_token[:20]}...")
230         print(f" Refresh token: {refresh_token[:20]}...")
231         print(" Expected metrics: auth_flows_success, access_tokens_issued,")
232         print(" refresh_tokens_issued, active_tokens (access + refresh)")
233         wait_for_prometheus()
234         check_metric("oauth2_auth_flows_success_total")
235         check_metric("oauth2_refresh_tokens_issued_total")
236         check_metric("oauth2_pending_auth_codes")
237         return access_token, refresh_token
238     else:
239         print(f" ERROR: {response.status_code} --{response.text}")
240         return None, None
241
242
243 def test_refresh_token_flow(refresh_token: str) -> str | None:
244     """
245     Tests the refresh_token flow.
246     Expected metrics incremented:
247         - auth_flows_success{grant_type="refresh_token"}
248         - tokens_revoked{token_type="refresh"} (old token revoked)
249         - tokens_revoked{token_type="access"} (old access token revoked)
250         - active_tokens decremented then incremented
251     Returns the new refresh_token for later use.
252     """
253     separator("TEST 5 --Valid refresh_token flow")
254     print(f" Using refresh token: {refresh_token[:20]}...")
255
256     response = requests.post(f"{BASE_URL}/token", data={
257         "grant_type": "refresh_token",
258         "refresh_token": refresh_token,
259         "client_id": WEB_APP_ID,
260         "client_secret": WEB_APP_SECRET
261     })
```

```
262
263     print(f" Response status: {response.status_code} (expected 200)")
264
265     if response.status_code == 200:
266         data = response.json()
267         new_refresh = data.get("refresh_token")
268         print(f" New refresh token: {new_refresh[:20]}...")
269         print(" Old tokens should be revoked (token rotation)")
270         print(" Expected metrics: tokens_revoked x2, active_tokens dec+inc")
271         wait_for_prometheus()
272         check_metric("oauth2_tokens_revoked_total")
273         check_metric("oauth2_auth_flows_success_total")
274         return new_refresh
275     else:
276         print(f" ERROR: {response.status_code} --{response.text}")
277         return None
278
279
280 def test_revoked_token_replay(refresh_token: str) -> None:
281     """
282     Tests using a revoked refresh token (token replay attack).
283     The token passed here should already be revoked from test 5.
284     Expected metrics incremented:
285         - revoked_token_usage
286         - auth_flows_failed{grant_type="refresh_token"}
287     """
288     separator("TEST 6 --Token replay attack (revoked token reuse)")
289     print(f" Trying to reuse already-revoked token: {refresh_token[:20]}...")
290
291     response = requests.post(f"{BASE_URL}/token", data={
292         "grant_type": "refresh_token",
293         "refresh_token": refresh_token,
294         "client_id": WEB_APP_ID,
295         "client_secret": WEB_APP_SECRET
296     })
297
298     print(f" Response status: {response.status_code} (expected 400)")
299     if response.status_code == 400:
300         print(" Server correctly rejected the revoked token")
301         print(" Expected metrics: revoked_token_usage incremented")
302     else:
303         print(f" Unexpected status: {response.status_code}")
304
305     wait_for_prometheus()
306     check_metric("oauth2_revoked_token_usage_total")
307
308
309 def test_auth_code_reuse(auth_code: str) -> None:
310     """
311     Tests reusing an already-consumed authorization code.
312     Expected metrics incremented:
313         - auth_code_reuse
314         - auth_flows_failed{grant_type="authorization_code"}
```

```
    """
    separator("TEST 7 --Auth code replay attack (code reuse)")
    print(f" Trying to reuse already-used auth code: {auth_code[:20]}...")

    response = requests.post(f"{BASE_URL}/token", data={
        "grant_type": "authorization_code",
        "code": auth_code,
        "redirect_uri": "http://localhost:8080/callback",
        "client_id": WEB_APP_ID,
        "client_secret": WEB_APP_SECRET
    })

    print(f" Response status: {response.status_code} (expected 400)")
    if response.status_code == 400:
        print(" Server correctly rejected the reused code")
        print(" Expected metrics: auth_code_reuse incremented")
    else:
        print(f" Unexpected status: {response.status_code}")

    wait_for_prometheus()
    check_metric("oauth2_auth_code_reuse_total")


def test_parallel_requests() -> None:
    """
    Sends 20 parallel requests to trigger requests_in_progress > 0.
    Expected metrics visible during execution:
        - requests_in_progress{endpoint="/token"} > 0
    Note: this gauge goes up and down so fast it may still show 0
    after the requests complete. Check Grafana for the spike.
    """
    separator("TEST 8 --Parallel requests (requests_in_progress)")
    print(" Launching 20 parallel requests to /token...")

    results = []

    def make_request():
        response = requests.post(f"{BASE_URL}/token", data={
            "grant_type": "client_credentials",
            "client_id": SERVICE_ACCOUNT_ID,
            "client_secret": SERVICE_ACCOUNT_SECRET,
        })
        results.append(response.status_code)

    # Launch all threads simultaneously
    threads = [threading.Thread(target=make_request) for _ in range(20)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()

    success = results.count(200)
    failed = len(results) - success
```

```
    print(f" Completed: {success} successful, {failed} failed")
    print(" Check Grafana for requests_in_progress spike during execution")
    print(" Check request_duration histogram for response time distribution")
    wait_for_prometheus()
    check_metric("oauth2_requests_in_progress")
    check_metric("oauth2_request_duration_seconds_count")


def test_expired_token() -> None:
    """
    Tests using a completely fake/non-existent refresh token.
    This simulates what happens when someone tries an expired or
    completely fabricated token.
    Expected metrics incremented:
        - auth_flows_failed{grant_type="refresh_token"}
        - expired_token_usage (if the token exists but is expired)
        Note: for a fake token the server returns None from validate_refresh_token
        so expired_token_usage may not increment (token not found vs expired)
    """
    separator("TEST 9 --Expired/fake token usage")
    print(" Trying to use a completely fake refresh token...")

    response = requests.post(f"{BASE_URL}/token", data={
        "grant_type": "refresh_token",
        "refresh_token": "this_is_a_completely_fake_token_that_does_not_exist",
        "client_id": WEB_APP_ID,
        "client_secret": WEB_APP_SECRET
    })

    print(f" Response status: {response.status_code} (expected 400)")
    wait_for_prometheus()
    check_metric("oauth2_expired_token_usage_total")
    check_metric("oauth2_auth_flows_failed_total")


def print_full_metrics_summary() -> None:
    """Print all oauth2 metrics with their current values."""
    separator("FULL METRICS SUMMARY")
    print(" All oauth2 metrics current values:\n")

    response = requests.get(METRICS_URL)
    lines = [
        line for line in response.text.splitlines()
        if "oauth2" in line and not line.startswith("#")
    ]

    if lines:
        for line in lines:
            print(f" {line}")
    else:
        print(" No oauth2 metrics found. Is the app running?")

def test_client_enumeration() -> None:
```

```
    separator("TEST 10 --Client enumeration attack")
    print(" Trying to access with unknown client_ids...")

    fake_clients = [
        "admin", "root", "test", "app", "api",
        "mobile", "backend", "service", "user", "guest"
    ]

    for fake_client in fake_clients:
        response = requests.post(f"{BASE_URL}/token", data={
            "grant_type": "client_credentials",
            "client_id": fake_client,
            "client_secret": "any_secret"
        })
        print(f" client_id={fake_client}: status {response.status_code} (expected 401)")

    print("\n Expected: unknown_client_attempts incremented 10 times")
    wait_for_prometheus()
    check_metric("oauth2_unknown_client_attempts_total")

def test_credential_stuffing() -> None:
    separator("TEST 11 --Credential stuffing attack")
    print(" Trying leaked credentials with different usernames...\n")

    leaked_credentials = [
        ("john.doe", "password123"),
        ("jane.smith", "qwerty"),
        ("carlos.garcia", "123456"),
        ("maria.lopez", "letmein"),
        ("pedro.martinez", "welcome"),
        ("ana.rodriguez", "monkey"),
        ("luis.hernandez", "dragon"),
        ("sofia.gonzalez", "master"),
        ]

    for username, password in leaked_credentials:
        response = requests.post(f"{BASE_URL}/authorize", data={
            "client_id": WEB_APP_ID,
            "redirect_uri": "http://localhost:8080/callback",
            "response_type": "code",
            "scope": "read",
            "state": "test123",
            "action": "approve",
            "username": username,
            "password": password
        })
    print(f" username={username}: status {response.status_code} (expected 401)")

    print("\n Expected: user_auth_failed x8 con usernames distintos")
    print(" Expected: unique_failed_usernames_active sube a 8+")
    wait_for_prometheus()
    check_metric("oauth2_user_auth_failed_total")
    check_metric("oauth2_unique_failed_usernames_active")
```

```
474
475 def test_token_harvesting() -> None:
476     separator("TEST 12 --Token harvesting attack")
477     print(" Requesting 10 tokens without revoking any...\n")
478
479     print(" active_tokens BEFORE harvesting:")
480     check_metric("oauth2_active_tokens")
481     print()
482
483     for i in range(10):
484         response = requests.post(f"{BASE_URL}/token", data={
485             "grant_type": "client_credentials",
486             "client_id": SERVICE_ACCOUNT_ID,
487             "client_secret": SERVICE_ACCOUNT_SECRET,
488             "scope": "read"
489         })
490         if response.status_code == 200:
491             print(f" Token {i+1} acquired and stored (not revoked)")
492         else:
493             print(f" Token {i+1} FAILED: {response.status_code}")
494
495     print("\n active_tokens AFTER harvesting:")
496     wait_for_prometheus()
497     check_metric("oauth2_active_tokens")
498     check_metric("oauth2_tokens_revoked_total")
499
500 def test_client_enumeration() -> None:
501     separator("TEST 10 --Ataque de enumeración de clientes (Sostenido)")
502     print(" Enviando client_ids desconocidos masivamente para Grafana...")
503
504     fake_clients = ["admin", "root", "test", "app", "api", "cliente_falso"]
505
506     # Enviamos 200 peticiones en lugar de 10 para que el 'rate' suba
507     for _ in range(20):
508         for fake_client in fake_clients:
509             requests.post(f"{BASE_URL}/token", data={
510                 "grant_type": "client_credentials",
511                 "client_id": fake_client,
512                 "client_secret": "any_secret"
513             })
514
515     print("\n Tráfico generado. Revisa Grafana ahora.")
516     wait_for_prometheus()
517     check_metric("oauth2_unknown_client_attempts_total")
518 # --Main ------------------------------------------------------------------
519
520 def main():
521     print("\n" + "="*60)
522     print(" OAuth2 Metrics Verification Script")
523     print(" Make sure app.py is running on localhost:5001")
524     print("="*60)
525
526     # Verify the app is reachable before starting
```

```
    try:
        requests.get(METRICS_URL, timeout=3)
        print("\n App is reachable at localhost:5001")
    except requests.exceptions.ConnectionError:
        print("\n ERROR: Cannot connect to localhost:5001")
        print(" Make sure app.py is running with: uv run python app.py")
        return

    # Run all tests in order
    # Test 1 --valid client_credentials
    test_client_credentials_valid()

    # Test 2 --brute force M2M
    test_client_credentials_invalid()

    # Test 3 --brute force web login
    test_user_login_invalid()

    # Test 4 --full authorization_code flow
    # We need to save the auth code before it gets consumed
    # so we run the login step manually here to save it
    separator("TEST 4 --Valid authorization_code flow")
    print(" Step 1: Logging in to get authorization code...")
    response = requests.post(f"{BASE_URL}/authorize",
        data={
            "client_id": WEB_APP_ID,
            "redirect_uri": "http://localhost:8080/callback",
            "response_type": "code",
            "scope": "read",
            "state": "test123",
            "action": "approve",
            "username": VALID_USERNAME,
            "password": VALID_PASSWORD
        },
        allow_redirects=False
    )

    auth_code = None
    access_token = None
    refresh_token = None

    if response.status_code == 302:
        location = response.headers.get("Location", "")
        auth_code = location.split("code=")[1].split("&")[0]
        print(f" Authorization code obtained: {auth_code[:20]}...")

        # Exchange the code for tokens
        print(" Step 2: Exchanging auth code for tokens...")
        response2 = requests.post(f"{BASE_URL}/token", data={
            "grant_type": "authorization_code",
            "code": auth_code,
            "redirect_uri": "http://localhost:8080/callback",
            "client_id": WEB_APP_ID,
```

```
            "client_secret": WEB_APP_SECRET
        })

        if response2.status_code == 200:
            data = response2.json()
            access_token = data.get("access_token")
            refresh_token = data.get("refresh_token")
            print(f" Access token: {access_token[:20]}...")
            print(f" Refresh token: {refresh_token[:20]}...")
            wait_for_prometheus()
            check_metric("oauth2_auth_flows_success_total")
            check_metric("oauth2_refresh_tokens_issued_total")
            check_metric("oauth2_pending_auth_codes")

    # Test 5 --refresh_token flow (uses the refresh_token from test 4)
    new_refresh_token = None
    if refresh_token:
        new_refresh_token = test_refresh_token_flow(refresh_token)

    # Test 6 --token replay with the OLD refresh_token (now revoked)
    if refresh_token:
        test_revoked_token_replay(refresh_token)

    # Test 7 --auth code reuse with the code from test 4 (now consumed)
    if auth_code:
        test_auth_code_reuse(auth_code)

    # Test 8 --parallel requests
    test_parallel_requests()

    # Test 9 --fake token
    test_expired_token()

    # Test 10 - usuarios this_is_a_completely_fake_token_that_does_not_exist
    test_client_enumeration()
    test_credential_stuffing()
    test_token_harvesting()
    test_client_enumeration()


if __name__ == "__main__":
    main()
```

D Locust

```python
from locust import HttpUser, task, between


class OAuth2LoadTest(HttpUser):
    """
    Virtual Locust user for testing the OAuth2 server.

    This user simulates both legitimate traffic and controlled malicious
    traffic to observe the application behavior and its Prometheus metrics.

    Metrics covered:
        - auth_flows_started / auth_flows_success / auth_flows_failed
        - tokens_issued / access_tokens_issued / refresh_tokens_issued
        - http_requests / http_requests_token / http_requests_authorize /
          http_requests_admin
        - requests_in_progress / request_duration
        - active_tokens / tokens_revoked
        - pending_auth_codes
        - auth_code_reuse
        - revoked_token_usage
        - expired_token_usage (indirectly via invalid refresh token)
        - brute_force_attempts
        - user_auth_failed
        - active_clients

    Traffic distribution:
        Legitimate: 6+3+2 = 11/19 = 58% of traffic
        Malicious: 2+2+1+1+1 = 7/19 = 37% of traffic
        Monitoring: 1/19 = 5% of traffic
    """

    # Each virtual user waits between 3 and 7 seconds before the next task
    # This simulates real user think time between actions
    wait_time = between(3, 7)

    # --Client credentials (M2M flow) -------------------------------------
    VALID_CLIENT_ID = "service-account"
    VALID_CLIENT_SECRET = "6Q6TZj_hKy4xBnZhgBuIwdJ6lSaQbF-9XWR9S_BHthk"

    # --Web application client (authorization_code flow) ------------------
    WEB_CLIENT_ID = "web-app"
    WEB_CLIENT_SECRET = "Vh_lxCQeixB4jAraZRxcdOkmP43YOeyXAn7IVomdMmc"
    WEB_REDIRECT_URI = "http://localhost:8080/callback"

    # --Valid test users defined in app.py --------------------------------
    VALID_USERNAME = "testuser"
    VALID_PASSWORD = "testpass"

    # ========================================================================
```

```
    # LEGITIMATE TRAFFIC
    # ================================================================================
    @task(6)
    def valid_client_credentials(self):
        with self.client.post("/token", data={
            "grant_type": "client_credentials",
            "client_id": self.VALID_CLIENT_ID,
            "client_secret": self.VALID_CLIENT_SECRET,
            "scope": "read"
        }, catch_response=True) as response:
            if response.status_code == 200:
                response.success()
            else:
                response.failure(f"Valid client_credentials failed with {response.
                    status_code}")

    @task(3)
    def valid_authorization_code_flow(self):
        """
        Complete authorization_code flow: load login page -> login -> get code -> exchange
            for tokens.
        Without this task, pending_auth_codes would grow indefinitely during
        load tests because codes would be issued but never consumed.

        Metrics fired:
            - auth_flows_started / auth_flows_success {grant_type="authorization_code"}
            - tokens_issued{token_type="access"/"refresh", grant_type="authorization_code"}
            - access_tokens_issued / refresh_tokens_issued {grant_type="authorization_code"
                }
            - active_tokens{token_type="access"} / active_tokens{token_type="refresh"}
            - pending_auth_codes (incremented then decremented)
            - http_requests / http_requests_authorize / http_requests_token with status 200

        Probability: 16% (3 of 19)
        """
        # Step 1: GET /authorize to simulate a real user loading the login HTML page
        with self.client.get("/authorize", params={
            "client_id": self.WEB_CLIENT_ID,
            "redirect_uri": self.WEB_REDIRECT_URI,
            "response_type": "code",
            "scope": "read",
            "state": "locust-test"
        }, catch_response=True) as get_response:

            if get_response.status_code != 200:
                get_response.failure(f"GET /authorize failed with {get_response.status_code}
                    ")
                return
            get_response.success()

        # Step 2: POST /authorize with valid credentials -> get the auth code
        with self.client.post("/authorize", data={
            "client_id": self.WEB_CLIENT_ID,
```

```
            "redirect_uri": self.WEB_REDIRECT_URI,
            "response_type": "code",
            "scope": "read",
            "state": "locust-test",
            "action": "approve",
            "username": self.VALID_USERNAME,
            "password": self.VALID_PASSWORD
        }, catch_response=True, allow_redirects=False) as auth_response:

            # Server returns 302 with the code in the redirect URL
            if auth_response.status_code not in (200, 302):
                auth_response.failure(
                    f"Valid authorize failed with {auth_response.status_code}"
                )
                return
            # Extract the code from the Location header
            code = None
            location = auth_response.headers.get("Location", "")
            if "code=" in location:
                code = location.split("code=")[1].split("&")[0]

            if not code:
                auth_response.success()
                return
            auth_response.success()
        # Step 3: POST /token exchanging the code -> get access + refresh token
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
        }, catch_response=True) as token_response:
            if token_response.status_code != 200:
                token_response.failure(
                    f"Code exchange failed with {token_response.status_code}"
                )
                return
            token_response.success()
    @task(2)
    def valid_refresh_token_flow(self):
        """
        Complete refresh_token flow: get tokens via authorization_code,
        then renew them using the refresh token.
        Without this task, tokens_revoked and refresh_tokens_issued
        would always be zero during load tests.

        Metrics fired:
            - auth_flows_started / auth_flows_success {grant_type="refresh_token"}
            - tokens_issued / access_tokens_issued / refresh_tokens_issued
            - active_tokens (new tokens activated)
            - tokens_revoked (old refresh token revoked via token rotation)
            - http_requests_token with status 200
```

```
151
152         Probability: 11% (2 of 19)
153         """
154         # Step 1: Get a valid pair of tokens via authorization_code
155         code = self._get_authorization_code()
156         if not code:
157             return
158
159         with self.client.post("/token", data={
160             "grant_type": "authorization_code",
161             "client_id": self.WEB_CLIENT_ID,
162             "client_secret": self.WEB_CLIENT_SECRET,
163             "code": code,
164             "redirect_uri": self.WEB_REDIRECT_URI
165         }, catch_response=True) as token_response:
166
167             if token_response.status_code != 200:
168                 token_response.failure(
169                     f"Token acquisition for refresh failed with {token_response.status_code}
                         "
170                 )
171                 return
172
173             try:
174                 tokens = token_response.json()
175                 refresh_token = tokens.get("refresh_token")
176             except Exception:
177                 token_response.failure("Could not parse token response")
178                 return
179
180             token_response.success()
181
182         if not refresh_token:
183             return
184
185         # Step 2: Renew tokens using the refresh token
186         with self.client.post("/token", data={
187             "grant_type": "refresh_token",
188             "client_id": self.WEB_CLIENT_ID,
189             "client_secret": self.WEB_CLIENT_SECRET,
190             "refresh_token": refresh_token
191         }, catch_response=True) as refresh_response:
192
193             if refresh_response.status_code != 200:
194                 refresh_response.failure(
195                     f"Valid refresh_token failed with {refresh_response.status_code}"
196                 )
197                 return
198
199             refresh_response.success()
200
201     # ============================================================================
202     # MALICIOUS TRAFFIC / ATTACK SIMULATION
```

```
    # ============================================================================

    @task(2)
    def brute_force_api(self):
        """
        M2M brute force attack: tries to guess the client secret.
        The server should defend correctly returning 401.

        Metrics fired:
            - auth_flows_started / auth_flows_failed {grant_type="client_credentials"}
            - brute_force_attempts{client_id="service-account"}
            - http_requests / http_requests_token with status 401

        Probability: 11% (2 of 19)
        """
        with self.client.post("/token", data={
            "grant_type": "client_credentials",
            "client_id": self.VALID_CLIENT_ID,
            "client_secret": "wrong_secret_attempt"
        }, catch_response=True) as response:

            # 401 is the expected behavior, server defended correctly
            if response.status_code == 401:
                response.success()
            else:
                response.failure(f"Expected 401 but got {response.status_code}")

    @task(2)
    def brute_force_user_login(self):
        """
        Web brute force attack: tries to guess a user password.
        The action field is required for the app to reach the credential
        check. Without it the app fails before checking the password
        and user_auth_failed would never be incremented.

        Metrics fired:
            - user_auth_failed{username=..., client_id="web-app"}
            - http_requests / http_requests_authorize with status 401

        Probability: 11% (2 of 19)
        """
        with self.client.post("/authorize", data={
            "client_id": self.WEB_CLIENT_ID,
            "redirect_uri": self.WEB_REDIRECT_URI,
            "response_type": "code",
            "scope": "read",
            "state": "locust-test",
            "action": "approve", # required for app to reach login check
            "username": self.VALID_USERNAME,
            "password": "wrong_password_attempt"
        }, catch_response=True) as response:

            if response.status_code == 401:
```

```
                response.success()
            else:
                response.failure(f"Expected 401 but got {response.status_code}")

    @task(1)
    def auth_code_replay_attack(self):
        """
        Authorization code replay attack: tries to use the same auth code twice.
        The first exchange should succeed (200), the second should fail (400)
        because the code is already marked as used in the database.
        This is more realistic than using an access token as refresh token.

        Metrics fired:
            - pending_auth_codes (incremented then decremented on first exchange)
            - auth_code_reuse (incremented on second exchange attempt)
            - http_requests_token with status 200 then 400

        Probability: 5% (1 of 19)
        """
        # Get a valid auth code
        code = self._get_authorization_code()
        if not code:
            return

        # First exchange: should succeed (200)
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
        }, catch_response=True) as first_response:

            if first_response.status_code != 200:
                first_response.failure(
                    f"First exchange failed unexpectedly with {first_response.status_code}"
                )
                return

            first_response.success()

        # Second exchange of the same code: should fail (400)
        # This fires auth_code_reuse in models.py -> consume_authorization_code()
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
        }, catch_response=True) as second_response:

            if second_response.status_code in (400, 401):
                second_response.success()
```

```
            else:
                second_response.failure(
                    f"Expected 400/401 on code replay but got {second_response.status_code}"
                )

    @task(1)
    def revoked_token_replay_attack(self):
        """
        Token replay attack: gets a valid refresh token, rotates it
        (which revokes the old one) and then tries to use the revoked token.
        This is more realistic than using an access token as refresh token
        because it simulates a real token theft scenario.

        Metrics fired:
            - tokens_revoked{token_type="refresh"} / tokens_revoked{token_type="access"}
            - active_tokens (decremented on revocation)
            - revoked_token_usage (on attempt to use the revoked token)
            - http_requests_token with status 400

        Probability: 5% (1 of 19)
        """
        # Step 1: Get valid tokens
        code = self._get_authorization_code()
        if not code:
            return

        refresh_token = None
        with self.client.post("/token", data={
            "grant_type": "authorization_code",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "code": code,
            "redirect_uri": self.WEB_REDIRECT_URI
        }, catch_response=True) as token_response:

            if token_response.status_code != 200:
                token_response.failure(
                    f"Token acquisition failed with {token_response.status_code}"
                )
                return

            try:
                tokens = token_response.json()
                refresh_token = tokens.get("refresh_token")
            except Exception:
                token_response.failure("Could not parse token response")
                return

            token_response.success()

        if not refresh_token:
            return

```

```
        # Step 2: Rotate the token (this revokes the old refresh token)
        with self.client.post("/token", data={
            "grant_type": "refresh_token",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "refresh_token": refresh_token
        }, catch_response=True) as rotate_response:

            if rotate_response.status_code != 200:
                rotate_response.success()
                return

            rotate_response.success()

        # Step 3: Try to use the ORIGINAL refresh token (now revoked)
        # This fires revoked_token_usage in models.py -> validate_refresh_token()
        with self.client.post("/token", data={
            "grant_type": "refresh_token",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "refresh_token": refresh_token
        }, catch_response=True) as revoked_response:

            if revoked_response.status_code in (400, 401):
                revoked_response.success()
            else:
                revoked_response.failure(
                    f"Expected 400/401 on revoked token but got {revoked_response.
                        status_code}"
                )

    @task(1)
    def invalid_refresh_token(self):
        """
        Tries to use a completely fake refresh token.
        Tests the expired/invalid token error path.

        Metrics fired:
            - auth_flows_started / auth_flows_failed {grant_type="refresh_token"}
            - http_requests / http_requests_token with status 400

        Probability: 5% (1 of 19)
        """
        with self.client.post("/token", data={
            "grant_type": "refresh_token",
            "client_id": self.WEB_CLIENT_ID,
            "client_secret": self.WEB_CLIENT_SECRET,
            "refresh_token": "completely_fake_refresh_token_xyz123"
        }, catch_response=True) as response:

            if response.status_code == 400:
                response.success()
            else:
```

```
                response.failure(
                    f"Expected 400 for fake refresh token but got {response.status_code}"
                )

    # ================================================================================
    # MONITORING ENDPOINTS
    # ================================================================================

    @task(1)
    def list_admin_clients(self):
        """
        Queries the list of registered clients.
        Generates traffic on the admin endpoint metrics.

        Metrics fired:
            - http_requests / http_requests_admin with status 200
            - requests_in_progress / request_duration for /admin/clients

        Probability: 5% (1 of 19)
        """
        with self.client.get("/admin/clients", catch_response=True) as response:
            if response.status_code == 200:
                response.success()
            else:
                response.failure(f"/admin/clients GET failed with {response.status_code}")

    # ================================================================================
    # PRIVATE HELPER METHOD
    # ================================================================================

    def _get_authorization_code(self) -> str | None:
        """
        Performs the authorization step (POST /authorize) with valid credentials
        and returns the auth code extracted from the Location header.
        Returns None if the code could not be obtained.

        This helper is used by multiple tasks to avoid repeating the login
        logic in each one (DRY principle).

        Note: the action field is required for the app to reach the
        credential check. Without it the app fails before authenticating
        the user and no auth code is issued.
        """
        with self.client.post("/authorize", data={
            "client_id": self.WEB_CLIENT_ID,
            "redirect_uri": self.WEB_REDIRECT_URI,
            "response_type": "code",
            "scope": "read",
            "state": "locust-internal",
            "action": "approve", # required field
            "username": self.VALID_USERNAME,
            "password": self.VALID_PASSWORD
        }, catch_response=True, allow_redirects=False) as response:
```

```
467
468             if response.status_code not in (200, 302):
469                 response.failure(
470                     f"_get_authorization_code failed with {response.status_code}"
471                 )
472                 return None
473
474             # Extract the code from the Location header (302 redirect)
475             code = None
476             location = response.headers.get("Location", "")
477             if "code=" in location:
478                 code = location.split("code=")[1].split("&")[0]
479
480             response.success()
481             return code
482
483
484     @task(1)
485     def client_enumeration_attack(self):
486         """
487         Client enumeration attack: tries to access with unknown client_ids.
488         Fires: unknown_client_attempts
489         Probability: ~5% (1 of 20)
490         """
491         import random
492         fake_clients = [
493             "admin", "root", "test", "app", "api",
494             "mobile", "backend", "service", "user", "guest"
495         ]
496         fake_client = random.choice(fake_clients)
497
498         with self.client.post("/token", data={
499             "grant_type": "client_credentials",
500             "client_id": fake_client,
501             "client_secret": "any_secret"
502         }, catch_response=True) as response:
503             if response.status_code == 401:
504                 response.success()
505             else:
506                 response.failure(f"Expected 401 but got {response.status_code}")
507     @task(1)
508     def credential_stuffing_attack(self):
509         """
510         Credential stuffing attack: tries many different usernames
511         with common passwords. Different from brute force which
512         attacks one user with many passwords.
513         Fires: user_auth_failed, unique_failed_usernames
514         Probability: ~5% (1 of 20)
515         """
516         import random
517         # Simulates a list of leaked credentials from other sites
518         leaked_credentials = [
519             ("john.doe", "password123"),
```

```
            ("jane.smith", "qwerty"),
            ("carlos.garcia", "123456"),
            ("maria.lopez", "letmein"),
            ("pedro.martinez", "welcome"),
            ("ana.rodriguez", "monkey"),
            ("luis.hernandez", "dragon"),
            ("sofia.gonzalez", "master"),
        ]
        username, password = random.choice(leaked_credentials)

        with self.client.post("/authorize", data={
            "client_id": self.WEB_CLIENT_ID,
            "redirect_uri": self.WEB_REDIRECT_URI,
            "response_type": "code",
            "scope": "read",
            "state": "locust-test",
            "action": "approve",
            "username": username,
            "password": password
        }, catch_response=True) as response:
            if response.status_code == 401:
                response.success()
            else:
                response.failure(f"Expected 401 but got {response.status_code}")

    @task(1)
    def token_harvesting_attack(self):
        """
        Token harvesting: requests valid tokens but never uses or
        revokes them. active_tokens gauge will keep growing.
        Fires: active_tokens (keeps incrementing without decrementing)
        Probability: ~5% (1 of 20)
        """
        # Just request tokens without ever refreshing or revoking them
        # This simulates an attacker accumulating valid tokens
        with self.client.post("/token", data={
            "grant_type": "client_credentials",
            "client_id": self.VALID_CLIENT_ID,
            "client_secret": self.VALID_CLIENT_SECRET,
            "scope": "read"
        }, catch_response=True) as response:
            if response.status_code == 200:
                # Intentionally do nothing with the token
                # In a real attack the attacker would store it
                response.success()
            else:
                response.failure(f"Expected 200 but got {response.status_code}")
```